LEERPROJECT · 5E MIDDELBAAR

Fluo-app

Software maken stap voor stap

Functionele en technische analyse
met uitgewerkte UML-diagrammen.

Begrijpen · ontwerpen · bouwen · testen · opleveren

Wat moet de app doen?

Hoe maken we dat?

Hoe weten we dat het werkt?

Sint-Franciscus Evergem · 4 oktober 2026 · 54 pagina’s

Wat ga je in dit project leren?

Software maken begint niet met code typen. Eerst moet je begrijpen welk probleem je oplost. Daarna bepaal je wat de app moet doen, ontwerp je de schermen en gegevens, bouw je kleine onderdelen en controleer je of ze werken. In dit leerproject doorloop je die stappen met een herkenbare schoolapp.

De opdracht in één zin

De school wil bijhouden welke deelnemende leerlingen met helm en/of fluovestje naar school komen. Elk item levert één punt op. Medewerkers gebruiken dezelfde gegevens op hun telefoon. Een administrator bekijkt alle punten en trekt verschillende winnaars, waarbij meer punten meer kans geven.

Wat je op het einde kunt

Je kunt...	Je toont dat met...
Een wens omzetten in duidelijke afspraken.	Eisen, regels en eenvoudige gebruiksscenario’s.
Een werking visueel voorstellen.	Use-case-, activiteiten-, klassen-, sequentie- en toestandsdiagrammen in UML.
Een haalbare technische oplossing ontwerpen.	Schermen, tabellen, API-afspraken en algoritmen.
Een klein werkend onderdeel bouwen.	Een prototype dat gegevens ophaalt, verwerkt en bewaart.
Controleren en verbeteren.	Tests, foutverslagen en een korte demonstratie.

De tekst legt elk nieuw begrip uit en past het meteen toe. De voorbeelden zijn uitgewerkt; de kaders ‘Zelf proberen’ laten je een kleine variant maken. Je hoeft nog geen professionele analist te zijn. Basiskennis van variabelen, voorwaarden en herhalingen helpt bij het technische deel.

> Werk met fictieve leerlingen. Het doel is proeven van het hele ontwikkelproces. Een klasprototype wordt pas een echte schoolapp na extra controle van accounts, beveiliging, beheer en gegevensbescherming.

De route door deze cursus

Dit dossier bevat een functionele én technische analyse. Daarna volgen de eerste bouwstappen, testen en oplevering. De inhoud is in kleine stukken verdeeld, zodat je na elke stap iets tastbaars hebt om te bespreken.

1 Van opdracht naar een duidelijk probleem **4**

2 Functioneel en technisch: het verschil **9**

3 Eisen, regels en user stories **10**

4 UML: wie gebruikt de app en wat gebeurt er? **14**

5 Schermen en foutmeldingen ontwerpen **20**

6 Technische analyse en architectuur **25**

7 Gegevens, UML-klassen en database **27**

8 API, JSON en UML-sequentie **34**

9 Synchronisatie, algoritmen en toestanden **38**

10 Beveiliging en een bouwplan **42**

11 Testen, fouten herstellen en opleveren **45**

12 Opdrachten, beoordeling en voorbeeldantwoorden **49**

13 Begrippen en bronnen **53**

Zo werk je

Lees eerst de uitleg. Bekijk vervolgens hoe dezelfde stap voor de fluo-app is uitgewerkt. Maak de kleine opdracht en laat een klasgenoot uitleggen wat hij uit jouw resultaat begrijpt. Verschillende interpretaties zijn nuttige feedback: misschien ontbreekt een regel of staat iets nog te vaag geformuleerd.

> De blauwgroene kaders bevatten opdrachten of kernideeën. UML-diagrammen zijn echte modellen met een betekenis; de schermschetsen zijn voorbeelden van een mogelijke interface.

Alle stappen van software maken

Een idee groeit in kleine rondes

Bij softwareontwikkeling maak je een oplossing voor een gebruiker. Dat verloopt zelden perfect in één poging. Je bedenkt iets, werkt een stukje uit, toont het, krijgt feedback en past het aan. De stappen hieronder geven structuur aan dat werk. Je mag terugkeren naar een eerdere stap wanneer je iets bijleert.

Stap	Vraag bij de fluo-app	Resultaat
Begrijpen	Waarom wil de school punten bijhouden?	Een korte probleemomschrijving.
Analyseren	Wat moet de app precies doen?	Eisen, regels en scenario’s.
Ontwerpen	Hoe zien schermen, gegevens en communicatie eruit?	Wireframes, UML en technische keuzes.
Plannen	Wat bouwen we eerst en wie doet wat?	Een lijst kleine taken.
Bouwen	Hoe maken we een werkend onderdeel?	Code en een database.
Testen	Werkt het ook bij fouten en herhaling?	Testresultaten en verbeteringen.
Opleveren	Kan iemand anders het gebruiken?	Een demo en korte handleiding.
Onderhouden	Wat doen we bij fouten of nieuwe wensen?	Herstel, updates en aangepaste afspraken.

Voorbeeld: tijdens het tekenen van de registratie ontdek je dat ‘punt erbij’ niet duidelijk genoeg is. Telt opnieuw klikken nogmaals? Je gaat terug naar de analyse en kiest maximaal één registratie per leerling per dag. Daardoor veranderen de knopwerking en de database. Dat terugkeren is normaal: een vroeg gevonden onduidelijkheid bespaart later herstelwerk.

> **Zelf proberen ·** Kies één bekende app. Benoem een vraag die de makers vóór het programmeren moesten beantwoorden. Beschrijf welke fout je krijgt als ze die vraag overslaan.

De klantvraag samenvatten

Waarom eerst samenvatten?

Een klant vertelt meestal in gewone taal wat hij wil. Sommige dingen zijn duidelijk, andere ontbreken. Door de vraag eerst kort samen te vatten controleer je of je hetzelfde doel begrijpt. Je kiest daarbij nog geen programmeertaal of knopkleur.

Onze samenvatting voor de fluo-app

Medewerkers moeten op iPhone en Android deelnemers kunnen registreren. Ze kiezen leerjaar, klas en leerling. Een klaslijst toont de leerlingen alfabetisch op achternaam, hun punten en het klastotaal. Voor helm en fluo kunnen afzonderlijk punten worden toegekend. Een administrator krijgt een totaaloverzicht met filters en puntensortering en kan een gewogen trekking uitvoeren.

Wat staat vast?	Wat vullen we logisch in?
Helm geeft 1 punt; fluo geeft 1 punt.	Maximaal 2 punten per leerling per dag.
Gegevens moeten gedeeld zijn tussen toestellen.	We gebruiken één centrale server en database.
Alleen de administrator gebruikt de totaallijst en trekking.	We voorzien persoonlijke accounts met twee rollen.
Meer punten geven meer kans; geen dubbele winnaar.	Een winnaar verdwijnt uit de pot van die trekking.

We noemen ‘1ste, 2de...’ voortaan leerjaar. Het woord schooljaar gebruiken we voor een periode zoals 2026-2027. De punten tellen binnen een campagne, bijvoorbeeld een herfstactie. Zo heeft een teller altijd een duidelijke periode.

> **Zelf proberen ·** Schrijf de opdracht in maximaal drie zinnen voor iemand die de app nog nooit heeft gezien. Vermeld het doel, de gewone gebruiker en de extra taak van de administrator.

Wie is betrokken?

Een gebruiker is iemand die de app bedient

Niet iedereen die in de database staat, gebruikt de app. Een leerling krijgt punten, maar meldt zich in dit ontwerp niet aan. Een medewerker doet de registratie. De administrator beheert het overzicht en trekt winnaars. Dit onderscheid is belangrijk bij het ontwerpen van schermen en rechten.

Betrokkene	Wat doet die?	Wat heeft die nodig?
Medewerker	Observeert en registreert helm/fluo.	Een snelle, duidelijke klaslijst en betrouwbare opslag.
Administrator	Controleert leerlingen, totalen en trekking.	Filters, sortering, campagnebeheer en een vaste uitslag.
Leerling	Neemt deel en kan een prijs winnen.	Een begrijpelijke uitleg over punten en kansen.
Opdrachtgever	Vraagt en beoordeelt de oplossing.	Een app die de afgesproken taken uitvoert.
Ontwikkelaar / ICT	Bouwt en houdt het systeem werkend.	Duidelijke afspraken, tests en beheerinformatie.

Twee rollen zijn voldoende

Een rol is een groep toegangsrechten. De medewerker mag registreren en klaspunten bekijken. De administrator mag dat ook, plus beheer en trekking. We maken geen tien verschillende rollen voor dit leerproject. Wel schrijven we duidelijk op welke handelingen alleen voor de administrator bestemd zijn.

In een klassikale demo kan de leerkracht twee vooraf ingestelde fictieve accounts voorzien. Een rolkeuze in een scherm is handig voor een lokale schets, maar is geen veilige login voor een echte schoolapp. De server moet bij werkelijk gebruik bepalen welke rol de aangemelde persoon heeft.

> **Zelf proberen ·** Is een leerling hier een gebruiker of alleen een betrokkene? Leg uit wat er aan de analyse verandert als leerlingen later zelf hun punten mogen raadplegen.

Onduidelijkheden logisch oplossen

Van een vaag woord naar een afspraak

Een analist stelt vragen omdat software exacte beslissingen nodig heeft. ‘Punten bijhouden’ zegt bijvoorbeeld niet hoe vaak dat kan. We hoeven het leerproject niet ingewikkeld te maken: we kiezen eenvoudige afspraken die bij het doel passen en voeren die overal consequent uit.

Vraag	Onze keuze	Waarom?
Telt elke klik opnieuw?	Nee. Eén toestand per leerling, campagne en dag.	Een dubbelklik mag geen extra beloning geven.
Moet iedereen deelnemen?	De beheerder bepaalt de deelnemers; standaard is de leerlingenlijst opgenomen.	Registreren en loten gebruiken dezelfde duidelijke selectie.
Kan een fout hersteld worden?	Ja: vandaag door een medewerker, eerdere dagen door admin zolang de campagne open is.	Een verkeerde keuze mag niet voor altijd vaststaan.
Kan iemand later opnieuw winnen?	Ja, in een nieuwe trekking.	‘Slechts één keer’ geldt binnen één trekking.
Werkt invoer zonder internet?	Niet in de basisversie.	Een online oplossing is eenvoudiger en deelt meteen dezelfde gegevens.
Welke periode telt mee?	De geselecteerde campagne.	Geen onduidelijke teller die jaren blijft doorlopen.

Een keuze is geen natuurwet. De school kan een andere afspraak willen. Dan zoeken we uit welke eisen, gegevens en tests mee veranderen. Maximaal één registratie per week in plaats van per dag verandert bijvoorbeeld de unieke sleutel en de berekening van punten.

> **Zelf proberen ·** Bedenk één extra vraag die je aan de organisator zou stellen. Geef zelf een eenvoudige keuze en leg uit welk onderdeel van de app daardoor duidelijker wordt.

De opdracht begrenzen

Scope betekent: wat hoort erbij?

Een klein project kan blijven groeien door extra ideeën. Voor je begint, leg je vast wat je eerste versie bevat. Zo weet de opdrachtgever wat hij krijgt en weet het team waarop het beoordeeld wordt. Het doel is een klein volledig traject, geen verzameling half afgewerkte extra functies.

Dit bouwen we	Dit bouwen we nu niet
Webapp voor telefoon en computer.	Aparte iPhone- en Androidapps in appstores.
Leerjaar, klas, leerling en helm/fluo.	Automatische herkenning via camera of GPS.
Punten, klastotalen en correcties.	Extra bonuspunten of uitgebreide statistieken.
Adminlijst, filters en sortering.	Leerlingportaal en oudermeldingen.
Gewogen trekking met vaste uitslag.	Voorraadbeheer voor prijzen.
Eenvoudige deelnemerslijst en campagne.	Automatische koppeling met alle schoolplatformen.

Een klasprototype opbouwen

Begin met zes fictieve leerlingen in twee klassen en één campagne. Dat is genoeg om de navigatie, puntensom en trekking te testen. Daarna kun je meer leerlingen toevoegen. Een grote echte dataset is niet nodig om het analyseproces te begrijpen en maakt beginnersfouten moeilijker zichtbaar.

De documentatie beschrijft de hele basisapp. Tijdens de lessen bouwt het team haar in kleine stukken. De leerkracht kan login of serverbasis aanleveren wanneer de groep nog weinig webervaring heeft. Leerlingen blijven wel uitleggen wat die onderdelen doen en welke gegevens erdoor gaan.

> **Zelf proberen ·** Een collega vraagt tijdens de bouw ook een chatfunctie. Hoort die bij deze scope? Formuleer een kort antwoord dat verwijst naar het oorspronkelijke doel.

Functioneel of technisch?

Functionele analyse: wat moet gebeuren?

De functionele analyse beschrijft de werking voor de gebruiker: taken, regels, invoer, uitvoer en fouten. Voorbeeld: een medewerker kan bij een leerling helm en fluo aanduiden. Beide items leveren samen twee punten op. Hiervoor hoef je nog niet te weten hoe een databasequery eruitziet.

Technische analyse: hoe maken we dat?

De technische analyse beschrijft hoe onderdelen samenwerken om die werking te realiseren. Voorbeeld: JavaScript verstuurt twee booleans naar een PHP-API. De server controleert de gebruiker en bewaart ze in één databasebestand of databaseserver. Daarna berekent hij de punten.

Uitspraak	Hoort vooral bij...
De leerlinglijst staat op achternaam gesorteerd.	Functionele analyse
De server gebruikt een ORDER BY in de query.	Technische analyse
Een leerling kan eenmaal per trekking winnen.	Functionele analyse
Na selectie verwijderen we de kandidaat uit de array.	Technische analyse
Bij slechte verbinding is duidelijk dat opslag niet bevestigd is.	Functionele analyse
We controleren response.ok en tonen een foutstatus.	Technische analyse

De twee analyses horen bij elkaar. Een technisch ontwerp zonder duidelijke werking kan het verkeerde product opleveren. Een functioneel verhaal zonder technische uitwerking kan onuitvoerbare beloften bevatten. We gebruiken dezelfde voorbeelden in beide delen zodat je de samenhang ziet.

> **Zelf proberen ·** Classificeer: ‘de beheerder filtert op klas’ en ‘de API ontvangt class_id=3’. Schrijf bij elke uitspraak zelf één zin uit het andere soort analyse.

De functionele eisen van de app

Een functionele eis is een concrete afspraak over wat de app kan. We geven elke eis een nummer. Dat nummer gebruiken we later bij taken en tests. De eisen hieronder vormen samen de eenvoudige basisversie.

ID	De app moet...
F01	Een medewerker of administrator aanmelden en diens rol herkennen.
F02	Leerjaren tonen en daarna de klassen van het gekozen leerjaar.
F03	Deelnemers alfabetisch op achternaam tonen, met persoonlijke campagnepunten en klastotaal.
F04	Helm en fluo onafhankelijk aanduiden en de dagtoestand bewaren: nul, één of twee punten.
F05	Dubbele opslag van dezelfde leerling op dezelfde dag zonder extra punten behandelen.
F06	Bevestigde gegevens op de verschillende verbonden toestellen beschikbaar maken.
F07	Een foutieve registratie volgens de rol- en datumregel laten corrigeren.
F08	Alleen admin een schoolbrede lijst met leerjaar-/klasfilters en sortering op punten geven.
F09	Alleen admin x verschillende winnaars gewogen volgens hun punten laten trekken.
F10	Een uitgevoerde trekking bewaren en bij herladen dezelfde uitslag tonen.
F11	Admin leerlingen, klassen en deelname laten beheren, met een eenvoudige import als start.
F12	Admin een campagne met datumbereik laten openen en definitief sluiten.

Wat maakt een eis goed?

Een eis is begrijpelijk en testbaar. ‘Het overzicht is goed’ is te vaag. ‘De administrator kan op klas filteren’ kun je wel controleren: geef de app leerlingen uit twee klassen en bekijk of alleen de gekozen klas verschijnt. Een eis schrijft nog niet de volledige code voor.

> **Zelf proberen ·** Kies F03. Schrijf één test om de sortering te controleren en één test om het klastotaal te controleren. Gebruik zelf vier fictieve leerlingen.

Kwaliteit is ook een afspraak

Niet-functioneel betekent niet onbelangrijk

Een niet-functionele eis beschrijft een kwaliteit of randvoorwaarde. De app kan de juiste punten berekenen en toch onbruikbaar zijn omdat knoppen te klein zijn of opslag erg traag is. We kiezen voor dit leerproject enkele eenvoudige, controleerbare doelen.

ID	Afspraak	Hoe controleer je dat?
N01	Het kernproces werkt op iPhone en Android zonder horizontaal scrollen.	Voer dezelfde taken uit op beide echte toestellen.
N02	Een normale opslag reageert binnen ongeveer 2 seconden op het afgesproken testnetwerk.	Meet 20 opslagen; onderzoek duidelijke uitschieters.
N03	Een bevestigde wijziging verschijnt binnen 5 seconden op een tweede actieve verbonden app.	Vergelijk twee toestellen met dezelfde klas open.
N04	Alle beheeracties worden door de server voor medewerkers geweigerd.	Test ook rechtstreeks de beheer-API, niet alleen het menu.
N05	Namen en acties zijn leesbaar; bediening werkt met toetsenbord en duidelijke tekstlabels.	Controleer Tab/Enter, contrast, grotere tekst en aan/uit-labels.

Maak ‘snel’ meetbaar

Zonder een testopstelling blijft ‘snel’ een mening. We noteren welk toestel, netwerk en aantal leerlingen we gebruiken. De waarden hierboven zijn doelen voor de oefenversie, geen gemeten resultaten. Werkt het netwerk niet, dan kan de app geen punten delen; ze moet dat eerlijk melden.

Kwaliteitseisen gaan niet altijd over techniek. Begrijpelijke foutmeldingen en een zichtbaar verschil tussen ‘bezig’ en ‘opgeslagen’ helpen de gebruiker juiste beslissingen nemen. Een klein project kan dus al aandacht geven aan gebruiksgemak en betrouwbaarheid.

> **Zelf proberen ·** Maak ‘de app is gemakkelijk’ testbaar. Bedenk een taak, een proefgebruiker en een criterium dat je werkelijk kunt waarnemen.

De regels achter de punten

De school bepaalt de betekenis

Een bedrijfsregel is een afspraak die de software uitvoert. De regel komt uit het schoolproces en blijft dezelfde, ook als we een andere programmeertaal gebruiken. Zonder zulke regels kunnen twee ontwikkelaars allebei iets logisch bouwen dat toch verschillend werkt.

Regel	Concrete betekenis
R01	Helm = 1 punt; fluo = 1 punt. Beide samen = 2. Geen van beide = 0.
R02	Per deelnemer, campagne en datum bestaat hoogstens één dagregistratie.
R03	Een nieuwe registratie kan alleen tijdens een open campagne en voor een deelnemende leerling.
R04	Een medewerker corrigeert vandaag; admin kan ook eerdere campagnedagen corrigeren zolang de campagne open is.
R05	Punten worden uit de registraties berekend, niet willekeurig door de browser ingesteld.
R06	Alleen positieve campagnepunten van actieve deelnemers geven loten.
R07	Na selectie verdwijnt de volledige leerling uit die trekking; geen dubbele winnaar.
R08	De campagne sluit vóór de trekking. Kandidaten, gewichten en uitslag worden bewaard.

Een ontbrekende registratie is geen waarneming

‘Niet geregistreerd’ betekent dat we nog niets hebben vastgelegd. ‘Helm nee, fluo nee’ betekent dat er wel een waarneming met nul punten is bewaard. Beide dragen nul bij aan de som, maar ze vertellen iets anders. De lijst kan daarom een aparte dagstatus tonen.

We gebruiken de schooldatum in Europe/Brussels. Een persoonlijk totaal loopt over de hele campagne. Een klastotaal telt punten die tijdens registratie bij die klas hoorden. Een klaswissel verandert de persoonlijke identiteit en eerdere waarnemingen niet.

> **Zelf proberen ·** Noor heeft al 4 punten. Vandaag draagt ze beide items. Daarna wordt dezelfde toestand opnieuw opgeslagen. Wat is haar totaal? Leg uit met R01 en R02.

User stories schrijven

Een klein verhaal met een reden

Een user story beschrijft wie iets wil doen en waarom. Een handige vorm is: ‘Als [rol] wil ik [taak], zodat [doel].’ De reden voorkomt dat we zomaar een knop bouwen zonder te weten waarvoor die dient. Een story blijft kort; de details staan in regels en acceptatiecriteria.

Story	Uitwerking voor de fluo-app
US1	Als medewerker wil ik een klas op leerjaar kiezen, zodat ik snel de juiste leerlingen vind.
US2	Als medewerker wil ik helm en fluo afzonderlijk aanduiden, zodat elke waarneming de juiste punten oplevert.
US3	Als medewerker wil ik een fout kunnen corrigeren, zodat een leerling geen onjuist totaal houdt.
US4	Als administrator wil ik op klas filteren en op punten sorteren, zodat ik de volledige stand kan controleren.
US5	Als administrator wil ik gewogen verschillende winnaars trekken, zodat meer punten meer kans geven en niemand dubbel wint.

Acceptatiecriteria maken de story precies

Voor US2 kiezen we: beide items kunnen onafhankelijk aan of uit; Opslaan bewaart één dagtoestand; ja/ja geeft twee dagpunten; opnieuw opslaan maakt geen extra dagrij; de app toont pas definitief succes na serverbevestiging. Deze criteria verbinden het korte verhaal met F04, F05 en F06.

Een story is geen volledige analyse op zichzelf. ‘Als administrator wil ik een eerlijke randomizer’ mist de betekenis van eerlijk, de selectieperiode en het aantal winnaars. Daarom combineren we stories met bedrijfsregels, scenario’s, diagrammen en tests.

> **Zelf proberen ·** Schrijf een user story voor het bekijken van persoonlijke punten in de klaslijst. Voeg twee acceptatiecriteria toe zonder programmeertaal of database te noemen.

UML-use-cases leren lezen

UML is een tekentaal voor software

UML staat voor Unified Modeling Language. Met afgesproken symbolen maken we zichtbaar wie iets doet, welke stappen volgen en welke gegevens samenhangen. UML is geen programmeertaal. Je diagram is een model: het toont de belangrijke informatie voor de vraag die je onderzoekt [S1].

Een use-casediagram toont taken en gebruikers

Symbool	Wat betekent het?	Voorbeeld
Poppetje	Een actor: een rol buiten het systeem.	Medewerker.
Ovaal	Een taak die het systeem ondersteunt.	Helm en fluo registreren.
Rechthoek	De grens van het systeem.	Wat binnen de fluo-app gebeurt.
Gewone lijn	De actor neemt deel aan die taak.	Medewerker kan registreren.
Lege driehoek	Een rol is een bijzondere vorm van een andere rol.	Administrator is ook medewerker.

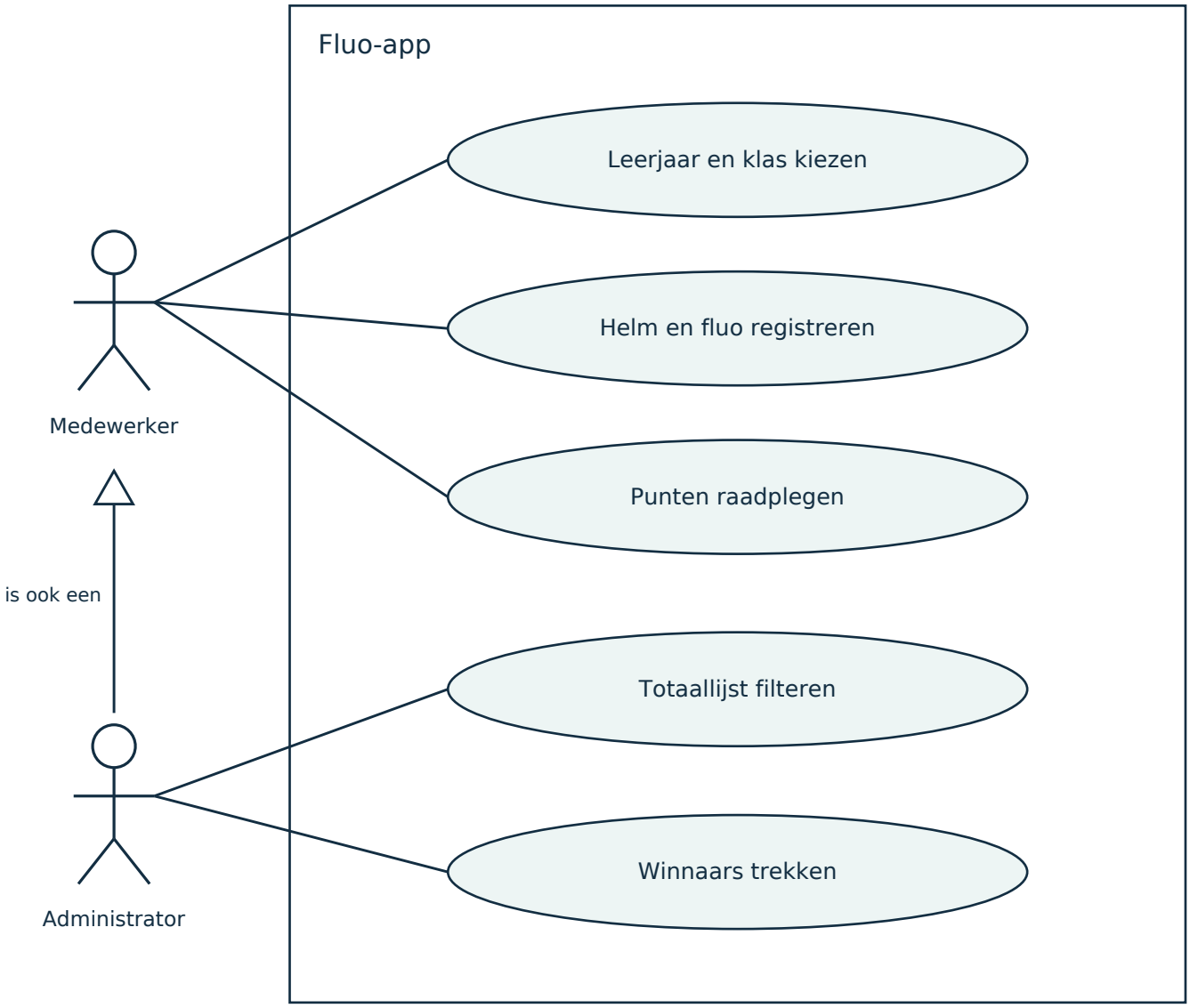
De administrator heeft alle gewone registratiefuncties en extra beheertaken. De lege driehoek wijst daarom naar Medewerker. Die driehoek is geen pijl voor gegevensverkeer. De leerling krijgt hier geen actorpoppetje: hij wordt geregistreerd, maar bedient de app niet.

Een use-casediagram toont niet de klikvolgorde en niet de database. Die vragen krijgen een eigen model. Door één vraag per diagram te beantwoorden blijft UML leesbaar, ook voor een kleine app.

> **Zelf proberen ·** Welke actor zou je toevoegen als leerlingen later zelf hun punten kunnen bekijken? Welke nieuwe use case hoort daarbij?

UML · De gebruikers en hun taken

Dit diagram werkt de belangrijkste taken uit. De lijn met de lege driehoek betekent dat een administrator ook alle taken van de medewerker kan uitvoeren.



De drie bovenste taken vormen het dagelijkse gebruik. De twee onderste zijn beheer. Aanmelden is een voorwaarde voor deze taken en is hier niet als aparte ovaal getekend. Het diagram is een overzicht van de kernfuncties, geen inventaris van ieder scherm of iedere beheerknop.

> **Zelf proberen ·** Zoek F04, F08 en F09 terug in het diagram. Vertel bij elke eis welke actor ermee verbonden is. Verklaar waarom de administrator niet naar alle ovalen een extra lijn nodig heeft.

Een registratie als gebruiksscenario

Een diagram geeft overzicht; tekst geeft details

Een use case beschrijft één doel van een actor. We leggen vast wat nodig is vóór de handeling, wat normaal gebeurt, welke fouten mogelijk zijn en wat het resultaat is. Daardoor wordt duidelijk wat de ontwikkelaar moet bouwen en wat de tester moet controleren.

Onderdeel	Registreren voor de fluo-app
Actor	Aangemelde medewerker of administrator.
Doel	De waarneming van één leerling voor vandaag bewaren.
Voorwaarden	Campagne open; leerling neemt deel; verbinding werkt.
Normaal verloop	Kies leerjaar, klas en leerling; duid helm/fluo aan; kies Opslaan; server bevestigt.
Resultaat	Eén dagregistratie staat bewaard en totalen zijn opnieuw berekend.
Afwijking	Ongeldige invoer, geen verbinding of verouderde toestand: geen stil succes en geen blinde overschrijving.

Concreet voorbeeld

Noor staat in 3A en heeft vijf campagnepunten. De medewerker opent Noor, zet Helm en Fluo beide aan en kiest Opslaan. De server bewaart de twee waarden samen. Vandaag levert twee punten op; Noor heeft nu zeven campagnepunten. De klaslijst toont het bevestigde resultaat.

Komt hetzelfde verzoek nogmaals binnen, dan verschijnt geen nieuwe dagrij. Wordt Fluo later uitgeschakeld als correctie, dan wordt de dagscore één en het campagnetotaal zes. Het normale scenario en de correctie gebruiken dus dezelfde betekenis van een dagregistratie.

> **Zelf proberen ·** Schrijf hetzelfde scenario voor Sara, die alleen een fluovestje draagt. Voeg één foutpad toe en beschrijf zowel de melding als het effect op de gegevens.

De trekking als gebruiksscenario

Doel: x verschillende winnaars

De administrator controleert eerst de campagne en sluit haar. Daarna kiest hij bewust het bereik en het aantal winnaars. De server berekent de kandidaten met positieve punten en bewaart hun gewichten. Pas na controle voert de administrator de trekking uit.

Stap	Wat gebeurt er?
1	Kies een gesloten campagne en bereik: school, leerjaar of klas.
2	Geef x op, bijvoorbeeld 2 winnaars.
3	Controleer de kandidaten, hun punten en het totaalgewicht.
4	Bevestig. De server trekt x verschillende leerlingen gewogen op punten.
5	De server bewaart de uitslag; het scherm toont de winnaars.
6	Bij herladen wordt diezelfde uitslag opnieuw getoond.

Wat kan fout gaan?

Er kunnen minder kandidaten zijn dan prijzen. Met drie kandidaten is x = 4 onmogelijk. Ook nul, negatieve getallen of een halve winnaar zijn ongeldig. De app toont een duidelijke melding en voert niets uit. Bij een dubbelklik mag dezelfde trekking niet twee verschillende uitslagen krijgen.

Voorbeeld: Noor heeft zeven punten, Milan vijf en Sara twee. Er zijn drie kandidaten en veertien loten. Twee winnaars kunnen Noor en Sara zijn. Noor verdwijnt na haar selectie volledig uit de pot; haar overige punten kunnen geen tweede prijs in dezelfde trekking opleveren.

> **Zelf proberen ·** Waarom moet de app zowel ‘3 kandidaten’ als ‘14 punten’ tonen? Leg uit met de vraag of er 4 winnaars gekozen kunnen worden.

UML-activiteiten leren lezen

Welke stappen volgen elkaar op?

Een activiteitendiagram beschrijft een verloop met acties, beslissingen en mogelijke herhaling. Het lijkt op een stroomdiagram en gebruikt UML-symbolen. We gebruiken het om te bekijken wat na een klik gebeurt en waar het proces kan afwijken.

Symbool	Betekenis	Voorbeeld
Gevulde cirkel	Start van het proces.	Medewerker begint met registreren.
Afgeronde rechthoek	Een actie.	Duid helm en fluo aan.
Pijl	De volgende stap.	Na kiezen volgt opslaan.
Ruit	Een beslissing met voorwaarden.	Is de invoer geldig?
Ruit als samenvoeging	Alternatieve routes komen samen.	Na een fout opnieuw naar invoer.
[ja] of [nee]	Voorwaarde op een uitgaande route.	Bij nee: toon foutmelding.
Cirkel met gevulde kern	Einde van deze activiteit.	Registratie is bevestigd.

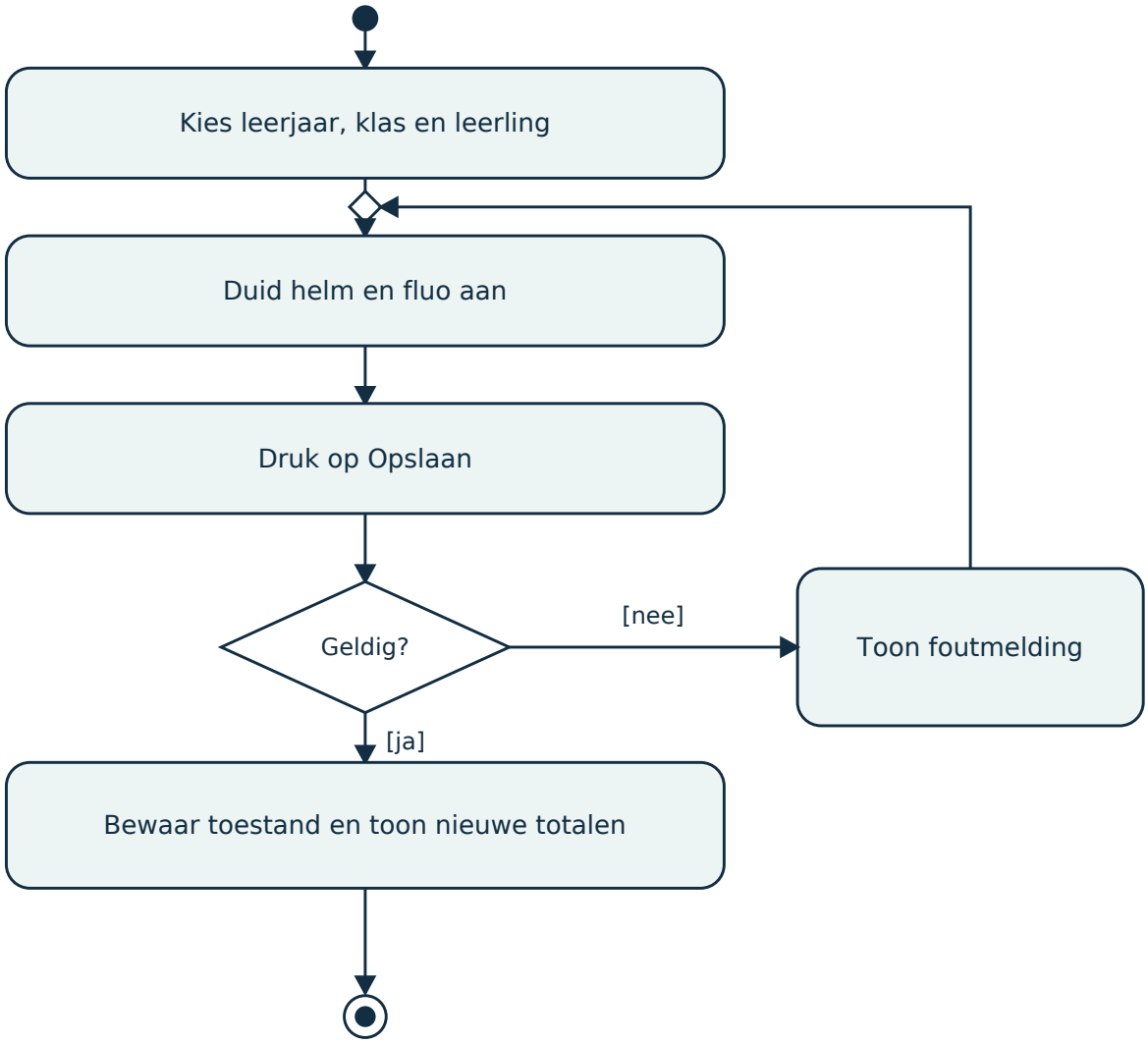
Een beslissing heeft betekenisvolle voorwaarden. ‘Geldig?’ gaat hier over deelnemer, datum, campagne en actuele versie. De nee-route vertelt de gebruiker wat hij moet herstellen en leidt terug naar de invoer. Ze eindigt niet met een valse bevestiging.

Een activiteitendiagram toont de volgorde van werk. Een use-casediagram toont de taken van rollen. Een sequentiediagram zal later laten zien welke softwareonderdelen bij die stappen met elkaar communiceren. Dezelfde opdracht kan dus meerdere diagrammen krijgen omdat we meerdere vragen beantwoorden.

> **Zelf proberen ·** Teken een heel kort activiteitendiagram voor aanmelden: naam invoeren, controleren en bij fout opnieuw proberen. Gebruik een startpunt, ruit en eindpunt.

UML · Een registratie doorlopen

Volg de pijlen vanaf de startcirkel. De kleine bovenste ruit voegt de normale route en de herstelroute samen. De grotere ruit beslist met de voorwaarden [ja] en [nee] welke route volgt.



De actie ‘Bewaar toestand’ omvat ook de servercontrole en bevestiging. Bij ongeldige invoer gaat de route terug naar het formulier. Een verbindingsfout krijgt een aparte melding; die is niet dezelfde beslissing als inhoudelijk ongeldige invoer.

> **Zelf proberen ·** Vertel het diagram hardop voor een leerling met helm zonder fluo. Loop het daarna door met een gesloten campagne. Op welke route verschijnt dan geen nieuw punt?

Van een taak naar schermen

Eerst de informatie, daarna de kleur

Een interface is wat de gebruiker ziet en bedient. Bij het ontwerp vraag je welke informatie nodig is om de taak correct uit te voeren. Daarna kies je waar die staat en welke actie het belangrijkste is. Een mooi scherm met onduidelijke informatie lost het probleem niet op.

Scherm	Nodige informatie	Hoofdactie
Aanmelden	Naam van app en schoollogin.	Aanmelden.
Leerjaar	Campagne, datum en leerjaren.	Leerjaar kiezen.
Klas	Leerjaar en passende klassen.	Klas kiezen.
Klaslijst	Namen, punten, dagstatus en klastotaal.	Leerling openen.
Leerlingdetail	Naam, klas, datum, helm en fluo.	Opslaan.
Beheer	Campagne, filters en schooltotalen.	Overzicht controleren.
Trekking	Bereik, kandidaten, gewicht en aantal winnaars.	Trekking bevestigen.

Een wireframe is een eenvoudige schets

Een wireframe mag met potlood worden gemaakt. Het toont de belangrijkste informatie en knoppen zonder volledige huisstijl. Zo kun je snel feedback vragen. Voor de fluo-app tekenen we een klaslijst en het detail voor één leerling. De naam en de datum blijven duidelijk zichtbaar om vergissingen te beperken.

Na een geslaagde opslag keert de app terug naar dezelfde klas en dezelfde positie. De medewerker hoeft niet voor elke leerling opnieuw zijn leerjaar te kiezen. Dat is een kleine ontwerpkeuze met veel effect op de dagelijkse snelheid.

> **Zelf proberen ·** Maak op papier drie schermen: leerjaar, klaslijst en leerlingdetail. Laat een klasgenoot zonder uitleg aanwijzen waar hij Noor met alleen een helm registreert.

Wireframe · Registreren op een telefoon

Links staat de klaslijst met campagnepunten. Rechts staat het detail van Milan met de toestand voor vandaag. De schermen zijn schetsen; ze leggen vast welke informatie zichtbaar moet zijn.



Totaal en dagscore zijn niet hetzelfde

Milan heeft links vijf campagnepunten. Rechts zijn helm en fluo aangevinkt: dat is twee punten voor vandaag. Na bevestigde opslag verwerkt de server die toestand in zijn campagnepuntentotaal. Als die dagtoestand al bestond, komt er geen tweede registratie bij.

Helm en Fluo zijn onafhankelijke keuzes. Opslaan bewaart beide samen. Tijdens de verzending toont de app ‘Bezig met opslaan’. Daarna volgt ‘Opgeslagen’ of een duidelijke foutmelding. Een groene knop op zichzelf bewijst niet dat er iets op de server bewaard is.

> **Zelf proberen ·** Teken het detail voor een leerling zonder helm maar met fluo. Geef aan waar je het verschil toont tussen niet opgeslagen en opgeslagen.

Wireframe · Het beheerscherm

De administrator moet kunnen zien welke selectie hij bekijkt. Daarom staan campagne, leerjaar, klas en sorteerrichting bovenaan. De trekking toont opnieuw haar eigen bereik en kandidaatcount.

Beheer · Herfstactie

Leerjaar: alle | Klas: alle | Punten: dalend

Leerling	Klas	Punten
Aerts, Noor	3A	7
De Smet, Milan	3A	5
Peeters, Sara	3A	2

Trekking: volledige school

3 kandidaten · 14 punten · 2 winnaars

Controleer de selectie en bevestig de trekking.

Filters veranderen de selectie

Een leerjaarfilter toont alleen leerlingen uit dat leerjaar. Een klasfilter beperkt die lijst verder. Wanneer de beheerder het leerjaar verandert, wordt een niet-passende klaskeuze leeggemaakt. Anders kan een verborgen combinatie ten onrechte een lege lijst opleveren.

Sorteren verandert de volgorde

Sorteren op punten verbergt geen leerlingen; het verandert hun plaats in de lijst. Bij gelijke punten gebruiken we achternaam en daarna voornaam. Zo verspringen leerlingen niet willekeurig wanneer de lijst opnieuw geladen wordt. De server sorteert de volledige selectie, niet alleen de toevallig zichtbare rijen.

Een trekking kan voor school, leerjaar of klas worden voorbereid. De administrator moet dat bereik daar expliciet controleren. Een filter dat hij eerder in een ander scherm gebruikte, mag niet ongemerkt de prijzenpot veranderen.

> **Zelf proberen ·** Beschrijf het verschil tussen filteren en sorteren met drie fictieve leerlingen. Wat verandert er wel en niet aan het aantal resultaten?

Invoer controleren en fouten uitleggen

Validatie betekent: is deze invoer toegestaan?

De app moet niet alles accepteren wat binnenkomt. De browser kan de gebruiker snel helpen, maar de server doet de definitieve controle. Een gebruiker kan immers zelf een verzoek aanpassen zonder de normale knoppen te gebruiken.

Invoer of situatie	Controle	Melding
Leerling-ID	Leerling bestaat en neemt deel.	‘Deze leerling neemt niet deel.’
Datum	Binnen campagne; medewerker gebruikt vandaag.	‘Je kunt deze datum niet registreren.’
Helm / fluo	Echte ja/nee-waarden, geen vrij puntengetal.	‘Ongeldige registratiegegevens.’
Aantal winnaars	Geheel getal van 1 tot kandidaatcount.	‘Kies maximaal 3 winnaars.’
Gesloten campagne	Geen nieuwe of gewijzigde registraties.	‘De campagne is gesloten.’
Versie verouderd	De huidige rij is intussen gewijzigd.	‘De registratie is veranderd. Bekijk de nieuwe toestand.’
Verbinding weg	Opslag niet bevestigd.	‘Geen bevestiging ontvangen. Controleer opnieuw.’

Een melding helpt bij de volgende handeling

‘Fout!’ zegt te weinig. Een goede melding benoemt wat misgaat en wat mogelijk is. Toon geen technisch wachtwoord, databasepad of complete serverfout aan de gebruiker. De ontwikkelaar kan technische informatie in een afgeschermd log terugvinden.

Maak ook lege toestanden begrijpelijk. ‘Geen deelnemers in deze klas’ is geen crash. ‘Geen open campagne’ is geen verdwenen leerlingenlijst. Het scherm legt de betekenis uit en geeft waar passend een volgende actie.

> **Zelf proberen ·** Verbeter deze melding: ‘ERROR 422’. Gebruik het voorbeeld van iemand die 5 winnaars vraagt terwijl er maar 3 kandidaten zijn.

De functionele analyse controleren

Kan iemand de werking voorspellen?

De functionele analyse is voldoende duidelijk wanneer een ander op basis ervan kan vertellen wat de app moet doen. Je hoeft nog geen werkende code te hebben. Gebruik je eisen, regels, scenario’s en schermschetsen samen om tegenstrijdigheden te vinden.

Controle	Toepassing op de fluo-app
Zijn rollen duidelijk?	Medewerker registreert; admin krijgt beheer en trekking erbij.
Hebben tellers een betekenis?	Persoonlijke punten en klastotaal horen bij een campagne.
Zijn herhaling en fouten beschreven?	Eén dagrij; correctie vervangt de toestand; geen stil succes bij netwerkverlies.
Kunnen schermen de taak uitvoeren?	Leerjaar, klas, leerling, keuzes en Opslaan zijn aanwezig.
Is de loting begrijpelijk?	Punten bepalen gewicht; winnaar verdwijnt; aantal kandidaten begrenst x.
Kunnen we de eisen testen?	Elke eis heeft een controlevoorbeeld met verwachte uitkomst.

Een uitgewerkt controlegeval

Gegeven: Noor heeft vier punten en nog geen registratie voor vandaag. Wanneer een medewerker helm en fluo bewaart, verwacht je zes punten. Wanneer hetzelfde verzoek opnieuw komt, blijft het zes. Wanneer Fluo daarna gecorrigeerd wordt naar nee, wordt het vijf. Een ontwerp dat hier iets anders doet, botst met onze regels.

Vraag een klasgenoot dit geval alleen met jouw analyse op te lossen. Laat hem ook de betekenis van een gesloten campagne uitleggen. Waar hij moet gokken, ontbreekt mogelijk een afspraak. Verduidelijk die afspraak voordat je de technische oplossing uitwerkt.

> **Zelf proberen ·** Maak zelf een controlecase met een niet-deelnemer en met een leerling die nul punten heeft. Mag de eerste registreren? Mag de tweede meeloten?

Nu kiezen we hoe we het maken

Dezelfde werking krijgt een technische vorm

We weten nu wat de app moet doen. De technische analyse bepaalt welke onderdelen dat samen uitvoeren. We kiezen een gewone webapp: de gebruiker opent een website op zijn telefoon. iPhone en Android krijgen dezelfde interface en verbinden met dezelfde server.

Onderdeel	Taak	Techniek in het leerproject
Frontend	Schermen tekenen en keuzes opvangen.	HTML, CSS en JavaScript.
Backend / API	Invoer en rechten controleren; punten en trekking uitvoeren.	PHP op één centrale server.
Database	Leerlingen, registraties en uitslagen bewaren.	SQLite voor het klasprototype.
Verbinding	Verzoeken en antwoorden uitwisselen.	HTTP met JSON; HTTPS bij hosting.

SQLite bewaart de database in een bestand op de server. Dat bestand staat niet op iedere telefoon. Ook met SQLite blijven de gegevens dus centraal gedeeld. Een andere relationele database kan later dezelfde rol overnemen; de analysemethode blijft hetzelfde.

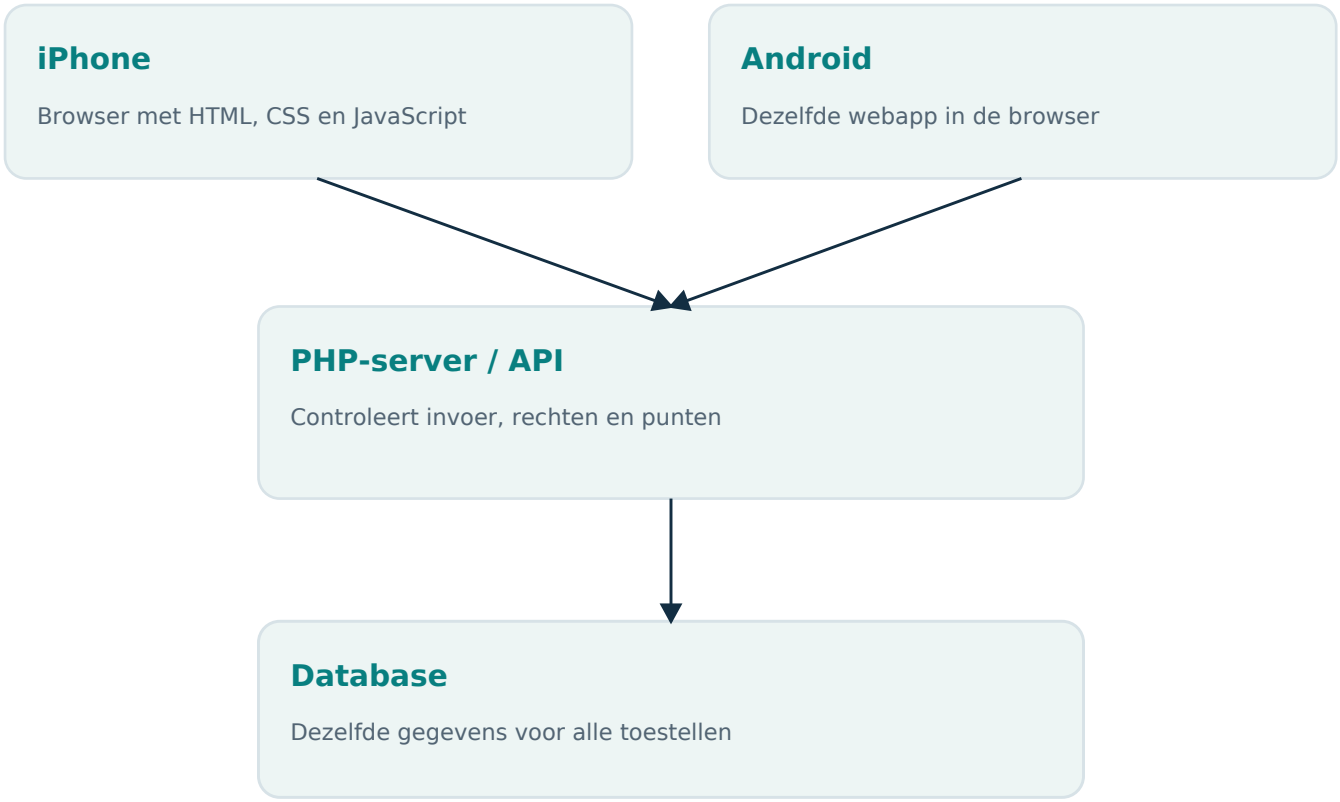
Waarom deze beperkte keuze?

Leerlingen kunnen zo de basis van scherm, server en opslag zien zonder eerst een groot framework te leren. Een bestaande Svelte- of andere omgeving kan dezelfde structuur gebruiken, maar is niet nodig om de principes te begrijpen. Het doel is de verantwoordelijkheid van elk onderdeel kunnen uitleggen.

> **Zelf proberen ·** Waar hoort de officiële puntenberekening: in de browser of op de server? Leg uit wat een gebruiker kan veranderen als we het totaal uit de browser zomaar geloven.

Architectuur · Eén gedeelde waarheid

Dit architectuurschema is geen UML-klassendiagram. Het toont waar de onderdelen draaien en hoe ze verbonden zijn. Beide telefoons lezen en wijzigen dezelfde database via de server.



Wat gebeurt bij Opslaan?

JavaScript stuurt de gekozen leerling en twee waarden naar de API. PHP controleert het verzoek en bewaart de dagtoestand. De database bevestigt de opslag. PHP antwoordt met de nieuwe toestand en totalen. JavaScript toont die bevestigde gegevens in het scherm.

De twee telefoons sturen geen gegevens rechtstreeks naar elkaar. Ze gebruiken dezelfde bron. Een teller die alleen in het geheugen van één telefoon staat, voldoet niet aan de opdracht. Die verdwijnt bij herladen of wijkt af van de teller op een ander toestel.

> **Zelf proberen ·** Volg in het schema een registratie van iPhone tot database en terug. Beschrijf daarna hoe Android die wijziging te zien krijgt.

Welke gegevens hebben we nodig?

Zoek objecten en eigenschappen

Lees de opdracht en zoek de dingen waarover we informatie bewaren: klas, leerling, campagne, registratie en trekking. Een eigenschap vertelt iets over één ding. Voor een leerling zijn dat bijvoorbeeld naam en deelname. Een unieke ID zegt welke leerling het is, ook bij gelijke namen.

Gegeven	Datatype	Waarom?
Leerling-ID	Integer	Een uniek geheel getal om de leerling te herkennen.
Voornaam / achternaam	String	Tekst; niet automatisch uniek.
Leerjaar	Integer	Een niveau zoals 1, 2 of 3.
Helm / fluo / deelname	Boolean	Precies twee toestanden: ja of nee.
Registratiedatum	Date	Een kalenderdag zonder verwarring met de campagneperiode.
Puntentotaal	Integer, berekend	Som van de waarnemingen; geen vrij tekstveld.
Trekkingsuitslag	Lijst met leerlingen	Meerdere unieke winnaars in een vaste volgorde.

Bewaren of berekenen?

We bewaren helm en fluo per dag. De dagscore en het persoonlijke totaal worden daaruit berekend. Zo vermijd je dat een teller op vijf staat terwijl de registraties samen zeven punten opleveren. Eerst kies je de brongegevens; daarna bepaal je de afgeleide waarden.

Een naam is geen goede sleutel. Twee leerlingen kunnen Noor Peeters heten. Als je op naam opslaat, kun je hun punten door elkaar halen. Een stabiele ID blijft ook gelijk wanneer een naam wordt gecorrigeerd.

> **Zelf proberen ·** Geef voor ‘aantal winnaars’, ‘klasnaam’ en ‘campagne open’ een geschikt datatype. Leg uit waarom een Boolean geen vervanging is voor het volledige leerlingtotaal.

UML-klassen leren lezen

Een klasse beschrijft een soort ding

Een klassendiagram beschrijft soorten objecten, hun eigenschappen en relaties. Een klasse Leerling is niet één specifieke leerling, maar het model voor alle leerlingen. Noor is een voorbeeld van een object van die klasse. Je kunt zo'n diagram begrijpen zonder al objectgeoriënteerd te programmeren.

Notatie	Betekenis	Voorbeeld
Vak met klassenaam	Een soort object.	Leerling.
naam: String	Eigenschap en datatype.	achternaam: String.
dagPunten(): Integer	Een bewerking met resultaat.	Berekent helm + fluo.
Lijn tussen klassen	Objecten zijn aan elkaar gekoppeld.	Leerling hoort bij Klas.
1	Precies één object aan die kant.	Een leerling heeft één huidige klas.
0..*	Nul of meer objecten aan die kant.	Een klas heeft nul of meer leerlingen.
1..*	Eén of meer objecten.	Een geldige trekking heeft kandidaten.

Lees een relatie in twee richtingen

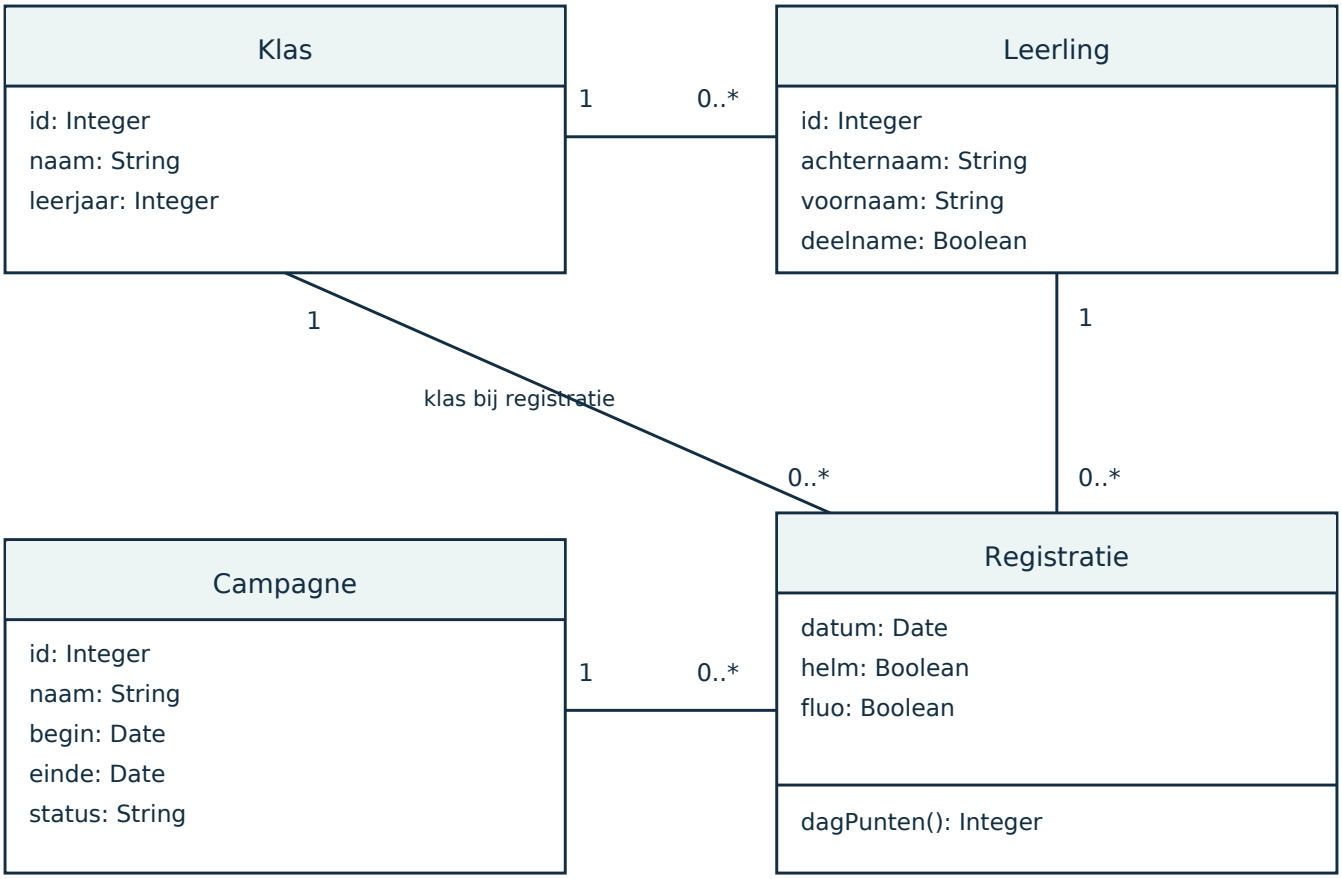
Bij Klas 1 - 0..* Leerling lees je: één klas kan nul of meer leerlingen bevatten; elke leerling hoort bij precies één huidige klas. De getallen heten multipliciteiten. Ze vertellen hoeveel verbindingen mogelijk zijn. Het sterretje betekent 'meerdere', niet een vaste hoeveelheid.

Een klassendiagram is nog geen database. Het beschrijft betekenis en samenhang. Later vertalen we klassen naar tabellen en relaties naar verwijzingen. Een berekende bewerking zoals dagPunten hoeft dus geen apart invulveld in de database te worden.

> **Zelf proberen ·** Teken twee klassen: Boek en Uitlening. Geef beide twee eigenschappen. Hoeveel uitleningen kan één boek door de tijd hebben? Zet passende multipliciteiten op de lijn.

UML · Klassen voor de registratie

Dit vereenvoudigde model toont de kerngegevens. De registratie koppelt één leerling, één campagne en de klas op het registratiemoment. De twee booleans bepalen de dagpunten.



De relaties lezen

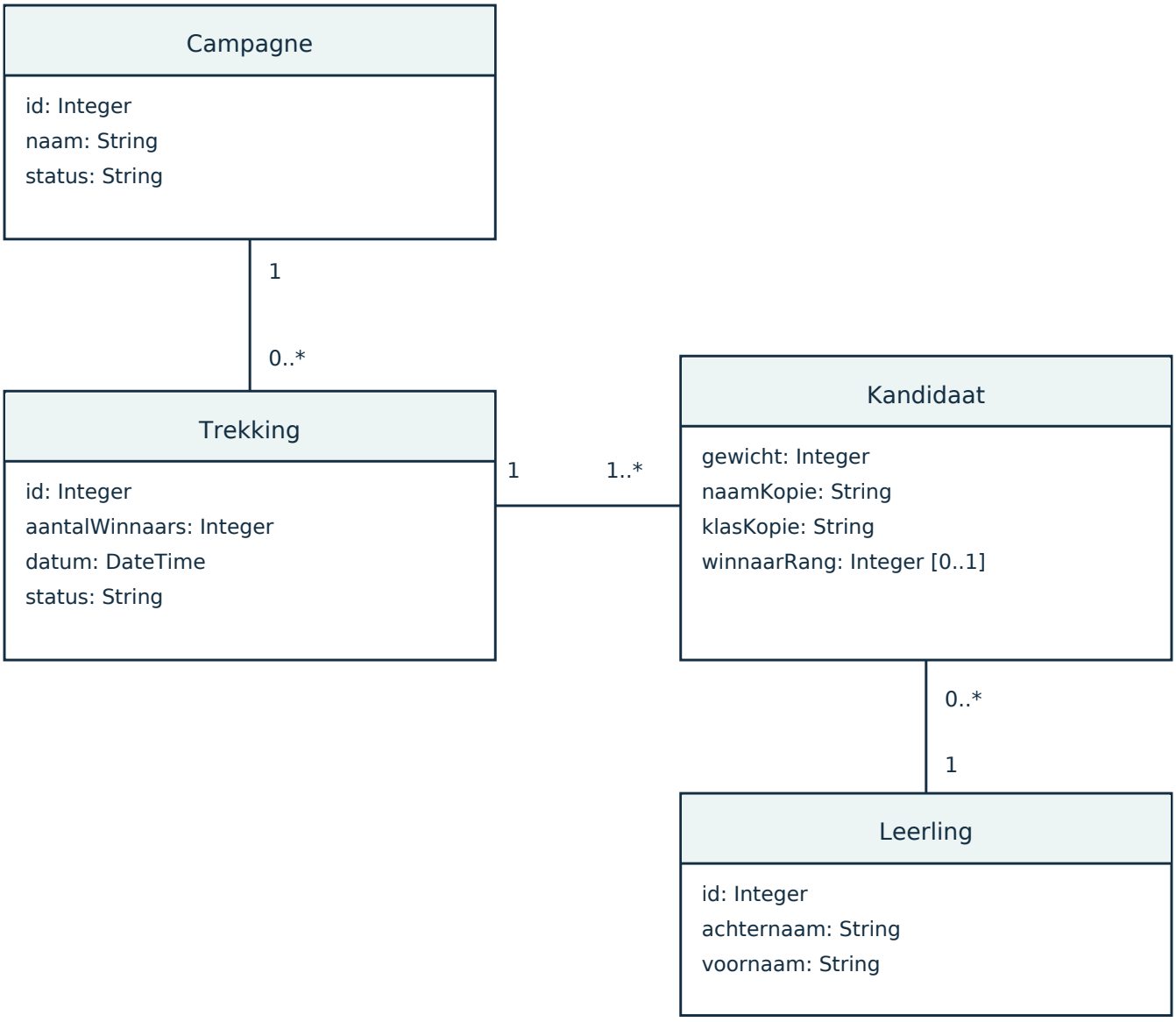
Een leerling heeft één huidige klas en kan meerdere registraties hebben. Elke registratie hoort bij één leerling en één campagne. Een campagne kan nul of meer registraties bevatten. De schuine relatie bewaart de klas die bij de waarneming hoorde, zodat een latere klaswissel oude klasbijdragen niet verplaatst.

De bewerking dagPunten() levert een Integer op. Voor helm = ja en fluo = nee is het resultaat één. Het diagram toont alleen de eigenschappen die nu nodig zijn. Technische velden, zoals versie en opslagmoment, voegen we bij de database en API toe.

> **Zelf proberen ·** Volg Noor met drie dagregistraties in dezelfde campagne. Hoeveel Leerling-objecten heb je? Hoeveel Registratie-objecten? Waarom niet voor elke dag een nieuwe leerling?

UML · Klassen voor de trekking

Een trekking bewaart kandidaten en gewichten. Een kandidaat is een leerling binnen één bepaalde trekking. Zo blijft achteraf duidelijk wie met hoeveel punten in de pot zat.



Een campagne kan meerdere trekkingen hebben. Elke geldige trekking heeft kandidaten, elk gekoppeld aan één leerling. Die leerling kan in verschillende trekkingen voorkomen. winnaarRang is optioneel: geen waarde betekent niet geselecteerd; 1 en 2 zijn de eerste en tweede winnaar. [0..1] betekent nul of één waarde. Naam en klas bewaren de oorspronkelijke context.

> **Zelf proberen ·** Er zijn drie kandidaten en twee winnaars. Welke drie mogelijke waarden kunnen de kandidaten na uitvoering voor winnaarRang hebben?

Van UML naar databasetabellen

Rijen, sleutels en verwijzingen

Een tabel bewaart objecten als rijen. Iedere rij heeft een primaire sleutel, meestal een ID. Een vreemde sleutel verwijst naar de ID in een andere tabel. Zo schrijft de registratie niet telkens een volledige leerlingnaam over, maar verwijst ze naar de juiste leerling.

Tabel	Belangrijkste velden
gebruiker	id, externeIdentiteit, naam, rol, actief. Voor de oefening mogen accounts vooraf ingesteld zijn.
klas	id, naam, leerjaar.
leerling	id, klasId, voornaam, achternaam, deelname.
campagne	id, naam, begin, einde, status, schooljaar.
registratie	id, leerlingId, campagneId, klasId, datum, helm, fluo, versie.
trekking	id, campagneId, bereik, aantalWinnaars, status, uitgevoerdOp.
kandidaat	trekkingId, leerlingId, gewicht, naamKopie, klasKopie, winnaarRang.

Een unieke combinatie is ook een regel

leerlingId + campagneId + datum moet in registratie uniek zijn. Anders kan dezelfde leerling dezelfde dag twee rijen krijgen. In kandidaat is trekkingId + leerlingId uniek. Zo kan een leerling niet dubbel in dezelfde pot belanden. Niet-lege winnaarRang is per trekking eveneens uniek.

Dit is een leermodel met beperkte historie: de deelname staat bij de leerling en we starten met één campagne. Het eerdere uitgebreide dossier kan extra deelnameperiodes, inschrijvingen en wijzigingslogs bevatten. Je hoeft die uitbreiding niet eerst te bouwen om de basis van relaties en sleutels te leren.

> **Zelf proberen ·** Welke velden in registratie zijn vreemde sleutels? Leg uit waarom helm geen vreemde sleutel is en leerlingId wel.

Een registratietabel in SQL

SQL is een taal om een relationele database te beschrijven en te bevragen. Dit SQLite-voorbeeld maakt de registratietabel. De tabellen leerling, campagne en klas moeten al bestaan. Het is een klein analysevoorbeeld, geen volledig installatiescript [S4].

```
PRAGMA foreign_keys = ON;
CREATE TABLE registratie (
  id INTEGER PRIMARY KEY,
  leerlingId INTEGER NOT NULL,
  campagneId INTEGER NOT NULL,
  klasId INTEGER NOT NULL,
  datum TEXT NOT NULL,
  helm INTEGER NOT NULL CHECK (helm IN (0, 1)),
  fluo INTEGER NOT NULL CHECK (fluo IN (0, 1)),
  versie INTEGER NOT NULL DEFAULT 1,
  UNIQUE (leerlingId, campagneId, datum),
  FOREIGN KEY (leerlingId) REFERENCES leerling(id),
  FOREIGN KEY (campagneId) REFERENCES campagne(id),
  FOREIGN KEY (klasId) REFERENCES klas(id)
);
```

Wat betekenen de beperkingen?

NOT NULL verplicht een waarde. CHECK beperkt helm en fluo tot nul of één. UNIQUE bewaakt de dagelijkse afspraak. FOREIGN KEY voorkomt verwijzen naar een niet-bestaande leerling, campagne of klas. In SQLite moet vreemde-sleutelcontrole voor de verbinding ingeschakeld zijn; daarom staat de PRAGMA bovenaan [S4].

De datum wordt als ISO-tekst bewaard, bijvoorbeeld 2026-10-06. De server controleert dat de tekst een echte kalenderdatum is en binnen de campagne valt. Een databasestructuur bewaakt veel regels, maar vervangt niet alle inhoudelijke controles.

> **Zelf proberen ·** Voorspel wat gebeurt bij helm = 3 en bij een tweede rij met dezelfde leerling, campagne en datum. Welke beperking weigert elk geval?

Totalen opvragen met SQL

De bron is de dagregistratie

Elke registratierij geeft helm + fluo punten. De server telt die waarden per leerling op voor de gekozen campagne. Een leerling zonder registraties moet toch in de klaslijst verschijnen, met nul punten. Daarom koppelen we de leerlinglijst met een LEFT JOIN.

```
SELECT l.id, l.achternaam, l.voornaam,
       COALESCE(SUM(r.helm + r.fluo), 0) AS punten
FROM leerling AS l
LEFT JOIN registratie AS r
  ON r.leerlingId = l.id
 AND r.campagneId = :campagneId
WHERE l.klasId = :klasId AND l.deelname = 1
GROUP BY l.id, l.achternaam, l.voornaam
ORDER BY l.achternaam, l.voornaam, l.id;
```

SUM telt op. COALESCE vervangt een ontbrekende som door nul. De campagnevoorwaarde staat bij de koppeling, zodat leerlingen zonder registratie niet verdwijnen. :campagneId en :klasId zijn parameters die de server gecontroleerd invult; we plakken geen willekeurige gebruikersinvoer aan de SQL-tekst.

Klastotaal

Het klastotaal is de som van registraties met de gekozen campagneId en bewaarde klasId. Daardoor blijven eerder verdiende punten bij de klas van de waarneming. Zonder klaswissels is dat ook de som van de persoonlijke totalen in de lijst. Na een wissel kan dat verschillen; het label moet dus ‘punten verdiend in deze klas’ betekenen.

> **Zelf proberen ·** Noor heeft dagpunten 2, 1 en 2. Milan heeft 1 en 0. Sara heeft nog geen rijen. Welke persoonlijke totalen toont de lijst? Wat is het klastotaal als niemand wisselde?

De API als afspraak tussen onderdelen

Een scherm vraagt, de server antwoordt

Een API is een afgesproken toegang tot functies en gegevens. De frontend hoeft niet te weten hoe de database is opgebouwd. Hij vraagt bijvoorbeeld een klaslijst of stuurt een waarneming. De backend controleert dat verzoek en antwoordt in een afgesproken formaat.

Verzoek	Doel	Wie mag dit?
GET /api/leerjaren	Leerjaren ophalen.	Beide rollen
GET /api/klassen?leerjaar=3	Passende klassen ophalen.	Beide rollen
GET /api/leerlingen?klasId=12&campagneId;=1	Klaslijst en punten ophalen.	Beide rollen
PUT /api/registraties	Dagtoestand bewaren of corrigeren.	Beide, volgens datumregel
GET /api/beheer/punten	Totaallijst filteren en sorteren.	Admin
POST /api/trekkingen	Trekking voorbereiden met kandidaten.	Admin
POST /api/trekkingen/8/uitvoeren	Voorbereide trekking 8 uitvoeren.	Admin
GET /api/trekkingen/8	Bestaande uitslag teruglezen.	Admin

GET is bedoeld om te lezen. PUT bewaart hier de gewenste registratietoestand. POST start een nieuwe voorbereiding of uitvoering. Een URL kiezen is geen beveiliging: de server controleert bij elke route de gebruiker en diens rechten.

De tabel is een contractvoorbeeld. Schooljaar, campagne en datum moeten waar nodig expliciet zijn. De programmeur mag geen stille standaard gebruiken waardoor een verzoek punten in de verkeerde campagne bewaart.

> **Zelf proberen ·** Welke route past bij ‘toon alle klassen van leerjaar 2’? Welke route mag een medewerker niet succesvol oproepen? Geef ook de verwachte reden.

JSON lezen en schrijven

Een eenvoudig formaat met namen en waarden

JSON is tekst waarmee programma’s gegevens uitwisselen. Een object gebruikt accolades en eigenschappen met waarden. Een lijst gebruikt vierkante haken. Strings hebben dubbele aanhalingstekens. Getallen en booleans hebben die niet.

```
{
  "leerlingId": 42,
  "campagneId": 1,
  "datum": "2026-10-06",
  "helm": true,
  "fluo": false,
  "verwachteVersie": 3
}
```

Dit verzoek zegt: bewaar voor leerling 42 in campagne 1 op die datum helm ja en fluo nee. Het zegt niet ‘voeg een punt toe’. Het zegt ook niet welk totaalcijfer de server moet geloven. De server berekent de gevolgen van de gewenste toestand.

Het antwoord van de server

```
{
  "opgeslagen": true,
  "versie": 4,
  "dagPunten": 1,
  "campagnePunten": 7
}
```

Een antwoord bevat de bevestigde toestand of een duidelijke fout. De versie is een technisch hulpmiddel om te zien of iemand met verouderde informatie werkt. Voor de eindgebruiker is vooral belangrijk dat hij weet of de opslag werkelijk is bevestigd.

> **Zelf proberen ·** Schrijf een JSON-verzoek voor Sara met beide items aanwezig. Welke waarden zijn getallen, welke tekst en welke booleans?

UML-sequenties leren lezen

Wie praat met wie, en wanneer?

Een sequentiediagram toont berichten tussen deelnemers, van boven naar beneden in de tijd. We gebruiken het voor het klikmoment Opslaan. De deelnemers zijn niet leerling en klas, maar de browser, de API en de database. Het diagram beantwoordt dus een andere vraag dan het klassendiagram.

Symbool	Betekenis
Vak bovenaan	Een deelnemer aan de communicatie, bijvoorbeeld api:Server.
Verticale stippellijn	De levenslijn van die deelnemer tijdens het scenario.
Horizontale pijl	Een verzoek of oproep naar een ander onderdeel.
Gestippelde terugpijl	Het antwoord op een eerder verzoek.
Hogere positie	Gebeurt eerder in het uitgewerkte scenario.

Het normale opslaanverloop

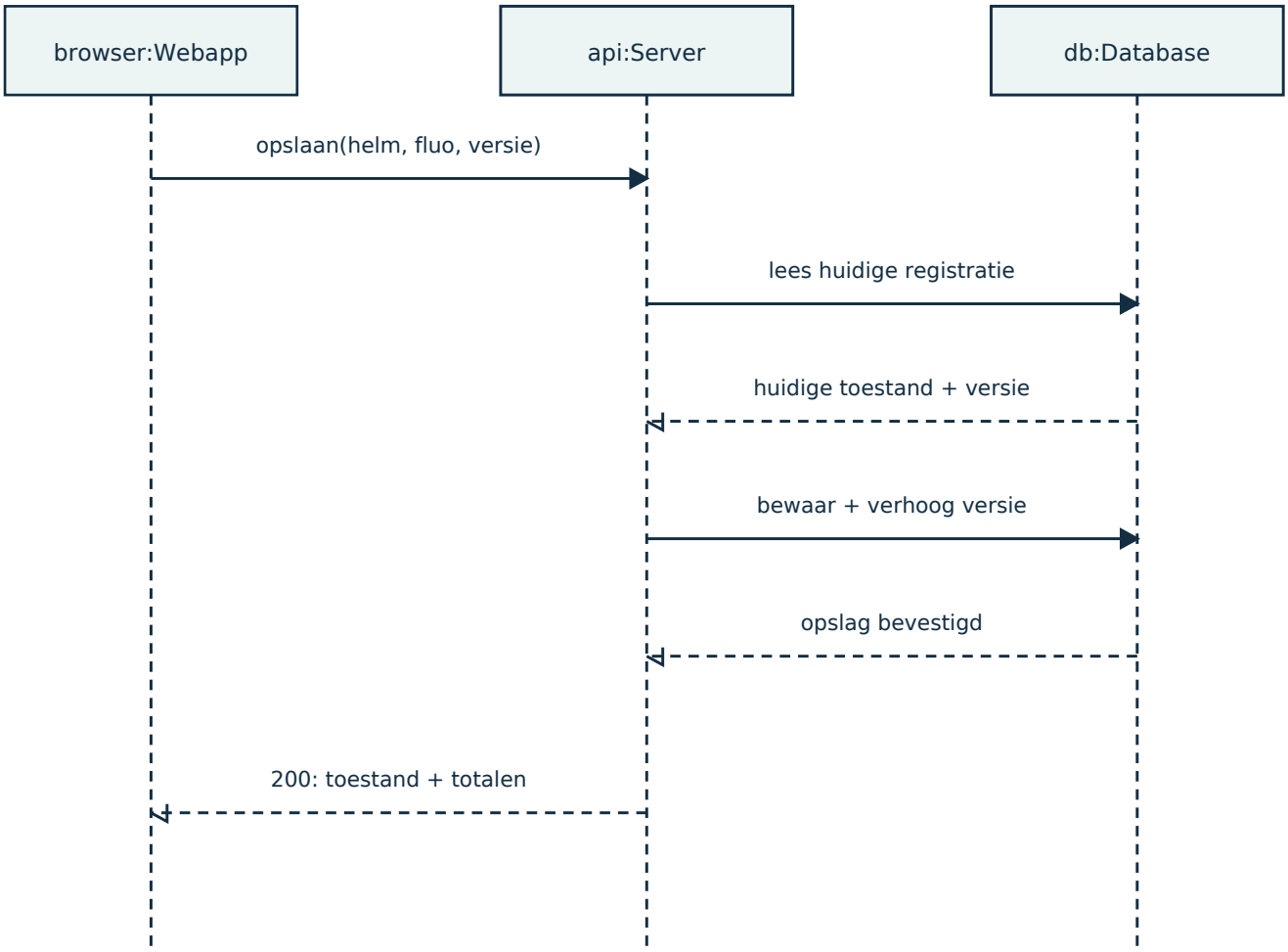
De browser verstuurt leerling, datum, helm, fluo en versie. De server controleert de gebruiker en invoer. Daarna leest hij de huidige registratie, vergelijkt de versie en bewaart de nieuwe toestand. De database bevestigt. De server stuurt de toestand en totalen terug. Pas dan toont de browser ‘Opgeslagen’.

Dit model helpt je onjuiste verantwoordelijkheden ontdekken. Een browser die rechtstreeks de database wijzigt, zou controle door de server omzeilen. Een succesmelding vóór het databaseantwoord zou opslag beloven die nog kan mislukken.

> **Zelf proberen ·** Welke twee deelnemers zouden communiceren bij het openen van een klaslijst? Heb je ook de database nodig? Beschrijf drie mogelijke berichten.

UML · Wat gebeurt bij Opslaan?

Lees van boven naar beneden. Na controle van gebruiker en invoer leest de server de huidige registratie en bewaart daarna de nieuwe toestand.



De bevestiging komt op het einde

De browser verstuurt een gewenste toestand en verwacht een antwoord. De server gebruikt de bestaande versie om ongewenst overschrijven te voorkomen. Na databasebevestiging krijgt de browser status 200 met toestand en totalen. Een HTTP-status is een code voor het resultaat van het verzoek.

Dit diagram toont alleen het normale pad. Bij een verkeerde versie wordt niet bewaard en antwoordt de server met een conflict. Bij een ongeldige sessie komt de server niet eens bij de getekende lees- en bewaarstappen. Een uitgebreider diagram kan die alternatieve routes toevoegen, maar voor de basis leren we eerst één leesbaar verloop.

> **Zelf proberen ·** Waar plaats je een conflictmelding als de versie niet klopt? Welke getekende berichten mogen dan niet meer gebeuren?

Synchroniseren en dubbel opslaan

Eén centrale stand, meerdere schermen

Beide telefoons lezen dezelfde servergegevens. Na eigen opslag gebruikt de app de bevestigde waarden in het antwoord. Een open klaslijst vraagt ongeveer elke drie seconden opnieuw de stand op. Bij terugkeer naar de app wordt meteen vernieuwd. Zo zien andere actieve toestellen de wijziging zonder rechtstreeks met elkaar te verbinden.

Wat als twee mensen tegelijk registreren?

Gebeurtenis	Resultaat
A en B lezen allebei versie 3.	Beiden starten vanuit dezelfde toestand.
A bewaart helm ja en fluo nee.	Server bewaart en verhoogt naar versie 4.
B bewaart een andere toestand met verwachte versie 3.	Server weigert blind overschrijven en geeft actuele versie 4 terug.
B bekijkt de actuele toestand en kiest opnieuw.	B kan bewust opslaan met versie 4.

Bij een dag zonder rij stuurt het scherm versie 0. De server maakt één rij met versie 1. De unieke dagcombinatie voorkomt een tweede invoeging. Controle en wijziging horen bij één transactie: ze slagen samen of worden samen teruggedraaid. Daardoor zit er geen half opgeslagen registratie tussen.

Antwoord verloren betekent niet zeker mislukt

De server kan al bewaard hebben terwijl het antwoord niet aankomt. De eenvoudige oefenapp vraagt daarom eerst de actuele dagtoestand opnieuw op. De gebruiker ziet geen definitief succes zonder bevestiging. Een uitgebreidere versie kan verzoek-ID’s gebruiken voor veilig automatisch herhalen; dat is aanvullende leerstof.

> **Zelf proberen ·** Waarom lost alleen de knop uitschakelen tijdens opslag niet alle dubbele invoer op? Denk aan twee verschillende telefoons.

Een algoritme voor puntentotalen

Een algoritme is een reeks duidelijke stappen

Voordat je code schrijft, kun je de oplossing in gewone taal of pseudocode beschrijven. Pseudocode lijkt op programmeercode, maar gebruikt begrijpelijke namen en hoeft niet uitvoerbaar te zijn. Daarmee kun je redeneren over voorwaarden en herhalingen zonder meteen op syntaxis vast te lopen.

```
totaal = 0
voor elke registratie van leerling in campagne:
    dagpunten = 0
    als registratie.helm aanwezig is:
        dagpunten = dagpunten + 1
    als registratie.fluo aanwezig is:
        dagpunten = dagpunten + 1
    totaal = totaal + dagpunten
geef totaal terug
```

Registratie	Dagpunten	Totaal na deze stap
Maandag: helm en fluo	2	2
Dinsdag: alleen helm	1	3
Woensdag: niets	0	3
Donderdag: alleen fluo	1	4

Hetzelfde principe in andere vormen

In JavaScript kun je de registraties doorlopen met een lus. In SQL kan SUM dezelfde optelling uitvoeren. Het algoritme blijft inhoudelijk hetzelfde. De technische vorm verandert, maar de bedrijfsregel niet. Bepaal steeds welke registraties bij de gekozen leerling en campagne horen.

Een correctie wijzigt een bestaande toestand. Ze voegt geen extra rij toe. Als woensdag van niets naar beide items verandert, wordt het totaal zes. Er zijn nog steeds vier dagregistraties. Dit voorbeeld laat zien waarom brongegevens betrouwbaarder zijn dan een losse knop die een teller verhoogt.

> **Zelf proberen ·** Doorloop de pseudocode voor vijf eigen waarnemingen. Laat een klasgenoot je tussenstanden controleren voordat je er code van maakt.

Een algoritme voor gewogen winnaars

Meer punten betekent meer loten

Neem A met zeven punten, B met één en C met twee. Er zijn tien punten. Kies een geheel willekeurig getal van 1 tot en met 10. A krijgt de getallen 1-7, B krijgt 8 en C krijgt 9-10. Zo heeft A bij de eerste keuze zeven keer zoveel kans als B.

Leerling	Gewicht	Getallen in de pot
A	7	1 t/m 7
B	1	8
C	2	9 t/m 10

```
overblijvers = kandidaten met punten > 0
winnaars = lege lijst
herhaal x keer:
    totaal = som van punten van overblijvers
    getal = willekeurig geheel getal van 1 tot totaal
    som = 0
    doorloop de overblijvers in een vaste volgorde:
        som = som + punten van deze leerling
        als getal <= som:
            voeg deze leerling toe aan winnaars
            verwijder deze leerling uit overblijvers
            stop deze doorloop
```

Na selectie verdwijnt de volledige leerling. Als A eerst wint, blijven drie punten over: B heeft getal 1 en C getallen 2-3. De kansverhouding verandert dus bij de volgende stap. Bij meerdere prijzen is de uiteindelijke kans om ergens te winnen niet simpelweg 7 op 1. Zijn er twee leerlingen en twee prijzen, dan winnen beiden zeker.

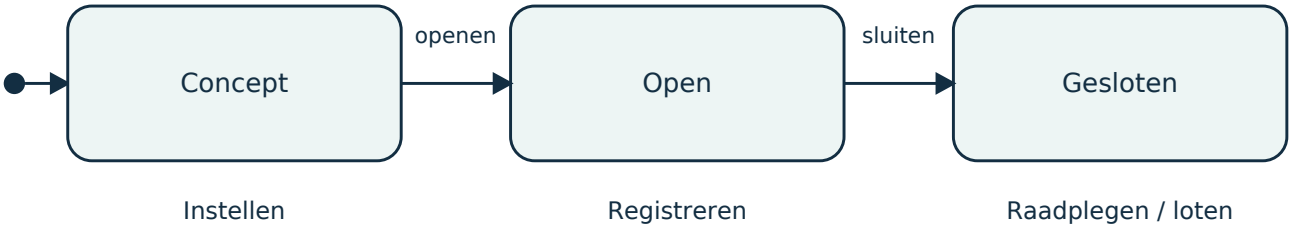
De server gebruikt een geschikte willekeurbron, bijvoorbeeld PHP random_int [S3]. Controleer vooraf dat x niet groter is dan het aantal kandidaten. Leerlingen met nul punten krijgen geen interval.

> **Zelf proberen ·** Gebruik eerst getal 8. Wie wint? Wie blijft over en wat zijn nu de intervallen? Gebruik daarna getal 9 in de kleinere pot.

UML · De toestand van een campagne

Sommige dingen mogen alleen in een toestand

Een toestandsdiagram beschrijft welke situaties een object kan hebben en welke gebeurtenissen een overgang veroorzaken. Voor een campagne kiezen we Concept, Open en Gesloten. Dat maakt duidelijk wanneer registreren is toegestaan. Het is een vereenvoudigd UML-toestandsmodel voor dit leerproject.



Toestand	Wel toegestaan	Niet toegestaan
Concept	Naam, periode en deelnemers klaarzetten.	Punten registreren of winnaars trekken.
Open	Registreren en fouten corrigeren volgens de rolregel.	Definitief loten uit veranderende punten.
Gesloten	Punten raadplegen en trekking uitvoeren.	Registraties toevoegen of aanpassen.

De administrator veroorzaakt de overgang openen en later sluiten. In onze basisversie is sluiten definitief. De app vraagt daarom een duidelijke bevestiging en raadt aan fouten eerst te controleren. Er staat bewust geen terugpijl van Gesloten naar Open.

Het diagram helpt ook de serverlogica: ieder opslagverzoek controleert of de campagne Open is. Een knop verbergen alleen is niet genoeg. Een medewerker kan het scherm al geopend hebben wanneer de administrator de campagne sluit.

> **Zelf proberen ·** Een leerlingdetail stond al open voordat de campagne werd gesloten. Mag Opslaan daarna nog lukken? Welke toestand bepaalt het antwoord?

Beveiliging en privacy in gewone taal

De server bewaakt de grenzen

De app bevat leerlinggegevens en verdeelt punten. Daarom controleren we wie gegevens mag lezen en wijzigen. Aanmelden vertelt wie je bent. Je rol bepaalt wat je mag doen. Een medewerker die zelf de trekkings-URL intypt, krijgt geen beheertoegang.

Eenvoudige afspraak	Waarom?
Persoonlijke accounts en servercontrole.	Rechten zijn niet afhankelijk van een verborgen knop.
Geen wachtwoorden in JavaScript of Git.	De browser en repository mogen geen geheimen bevatten.
Geparametriseerde SQL.	Gebruikersinvoer wordt als waarde behandeld, niet als databasecode.
Veilig weergeven van namen.	Tekst uit de database mag geen script worden in het scherm.
Alleen noodzakelijke gegevens.	Naam, klas en waarneming volstaan; foto en locatie zijn hier niet nodig.
Alleen fictieve leerlingen tijdens de les.	Een oefenproject hoeft geen echte leerlinggegevens te verwerken.
Back-up en beperkte bewaring.	Gegevens zijn herstelbaar en blijven niet zonder reden eindeloos staan.

Een prototype is een oefenversie

Je mag in de klas eerst met vaste testaccounts en een kleine dataset werken. Leg wel uit welke bescherming die versie al heeft en wat de leerkracht nog moet voorzien voor echt gebruik. Een lokaal rolkeuzescherm is geen productie-authenticatie. Voor de echte schoolapp gebruikt ICT de bestaande schoollogin en afgesproken privacyprocedures.

Veilige technische details worden later verder geleerd. De eerste belangrijke gewoonte is nu al: vertrouw geen puntengetal, rol of naam uit de browser zonder passende servercontrole. De frontend helpt de gebruiker; de server beslist wat geldig en toegestaan is.

> **Zelf proberen ·** Welke fout ontstaat als de browser zelf role='administrator' mag meesturen en de server dat gelooft? Formuleer de juiste controle in gewone taal.

Het bouwplan opdelen in kleine taken

Maak één bruikbaar stukje tegelijk

Een taak zoals ‘bouw de app’ is te groot. Een kleine taak heeft een zichtbaar resultaat en een controle. Begin bijvoorbeeld met een klaslijst uit fictieve gegevens. Voeg daarna één echte servervraag toe. Zo ontdek je fouten vroeg en blijft het werk overzichtelijk.

Taak	Werkend resultaat	Controle
B1	Leerjaar en klas kiezen.	Juiste klassen zichtbaar.
B2	Klaslijst met alfabetische namen.	Sortering en nulpunten kloppen.
B3	Helm/fluo in een detail invullen.	Vier toestanden mogelijk.
B4	Eén dagregistratie via API bewaren.	Herladen behoudt gegevens.
B5	Totalen en correctie toevoegen.	Geen dubbele punten.
B6	Adminoverzicht filteren en sorteren.	Beheer alleen voor admin.
B7	Gewogen trekking en vaste uitslag.	Unieke juiste kandidaten.
B8	Testen, uitleg en demo afronden.	Ander team kan de app gebruiken.

Bestanden hebben ook verantwoordelijkheden

Een eenvoudige structuur heeft index.html voor het scherm, styles.css voor de vormgeving, app.js voor interactie, een api-map voor PHP en een data-map voor de SQLite-database buiten de publieke webmap. Het README-bestand beschrijft hoe je het project start. De echte login kan door de leerkracht of ICT worden aangeleverd.

Verdeel in een duo niet alleen ‘jij doet voorkant, ik doe achterkant’. Spreek af welk JSON-verzoek en antwoord jullie delen. Anders kan de ene leerling helm schrijven terwijl de andere helmet verwacht. Het API-contract voorkomt zulke eenvoudige integratiefouten.

> **Zelf proberen** · Verdeel B1-B8 over een duo. Geef elke taak een eigenaar en één zin waarmee jullie bepalen dat ze klaar is.

Een eerste JavaScript-onderdeel

Begin met één verantwoordelijkheid

Deze uitgewerkte functie haalt een klaslijst op. Ze is een codefragment binnen de analyse, geen volledige app. De serverroute moet door het team of de leerkracht worden voorzien. Eerst begrijpt de leerling wat dit onderdeel verwacht en teruggeeft [S2].

```
async function haalLeerlingen(klasId, campagneId) {
  const params = new URLSearchParams({
    klasId: String(klasId),
    campagneId: String(campagneId)
  });
  const response = await fetch(
    '/api/leerlingen?' + params.toString()
  );
  if (!response.ok) {
    throw new Error('De klaslijst kon niet laden.');
  }
  return await response.json();
}
```

Wat gebeurt hier?

URLSearchParams maakt de queryparameters. fetch verstuurt het verzoek. await wacht op het resultaat binnen de asynchrone functie. response.ok vertelt of de HTTP-status succesvol is. response.json() leest het antwoord als gegevens. De schermcode kan daarna de leerlingen tekenen.

Een HTTP-fout zoals 403 laat fetch niet automatisch falen zoals een netwerkfout. Daarom controleren we response.ok expliciet [S2]. De code die deze functie oproept, vangt fouten op en toont een begrijpelijke melding. Ze toont geen oude klaslijst alsof die net succesvol vernieuwd werd.

> **Zelf proberen ·** Wat zijn invoer, verwerking en uitvoer van deze functie? Leg uit wat anders is als de API een geldige lege lijst teruggeeft of als het netwerk faalt.

Testen begint met een verwachting

Een klik is nog geen test

Bij een test weet je vooraf wat de uitkomst moet zijn. Daarna voer je de handeling uit en vergelijk je. ‘Ik heb op alles geklikt’ maakt niet duidelijk welke fout ontdekt zou worden. Gebruik concrete begingegevens, bijvoorbeeld Noor met vier punten en vandaag nog geen rij.

Onderdeel van een test	Uitgewerkt voorbeeld
ID	T04 · Beide items registreren.
Begintoestand	Noor is deelnemer, campagne open, totaal 4, vandaag nog niet geregistreerd.
Handeling	Open Noor, zet helm en fluo aan en sla op.
Verwacht	Eén dagrij, 2 dagpunten en totaal 6.
Werkelijk resultaat	Pas invullen nadat de app is gebouwd en de test is uitgevoerd.
Conclusie	Geslaagd of afwijking met een kort foutverslag.

Drie nuttige soorten gevallen

Normaal geval: een deelnemer met helm krijgt één dagpunt. Foutgeval: een medewerker probeert een beheeractie en wordt geweigerd. Grensgeval: het aantal winnaars is precies gelijk aan het aantal kandidaten. Je hebt alle drie nodig om meer te controleren dan het ideale pad.

Test op verschillende niveaus. Controleer eerst de formule voor dagpunten. Test daarna de API met database. Laat vervolgens een klasgenoot de complete taak op een telefoon uitvoeren. Een fout in de formule is anders dan een knop die de verkeerde leerling-ID verstuurt.

> **Zelf proberen ·** Ontwerp één normaal geval, één foutgeval en één grensgeval voor de trekking. Schrijf telkens op wat er na de test in de uitslag mag staan.

De tests van onze basisapp

Deze testgevallen zijn vooraf uitgewerkt. De kolom ‘verwacht’ is een afspraak, geen claim dat de software al getest is. Gebruik een frisse fictieve dataset per test waar dat nodig is.

Test / eis	Handeling	Verwacht
T01 / F01	Aanmelden als medewerker en als admin; ook zonder sessie proberen.	Juiste rol; zonder sessie geen leerlinggegevens.
T02 / F02	Leerjaar 3 kiezen.	Alleen passende klassen verschijnen.
T03 / F03	Klas openen met namen in willekeurige invoervolgorde.	Achternaam, voornaam en ID geven stabiele sortering; totalen kloppen.
T04 / F04	Vier combinaties helm/fluo bewaren.	Dagpunten 0, 1, 1 en 2.
T05 / F05	Dezelfde dag opnieuw opslaan, ook vanaf een tweede toestel.	Eén rij; geen extra punten door herhaling.
T06 / F06	Op A bewaren terwijl B dezelfde klas actief toont.	Bevestigde stand binnen 5 seconden op B.
T07 / F07	Beide items naar alleen helm corrigeren.	Totaal daalt met 1; onbevoegde datumcorrectie geweigerd.
T08 / F08	Adminlijst filteren en op punten sorteren; als medewerker de API proberen.	Juiste volledige selectie; medewerker geweigerd.
T09 / F09	Trekken met vaste testgetallen en x buiten bereik.	Juiste intervallen; unieke winnaars; ongeldige x geweigerd.
T10 / F10	Uitgevoerde trekking herladen en nogmaals dezelfde uitvoering aanroepen.	Dezelfde bewaarde uitslag.
T11 / F11	Dezelfde leerlingen opnieuw importeren; deelname wijzigen.	Geen dubbele identiteit; niet-deelnemer niet nieuw registreren.
T12 / F12	Open campagne sluiten tijdens invoer.	Na sluiting geen nieuwe punten; afgesloten stand leesbaar.

Zelf proberen · Kies één test en werk hem uit met exacte namen, beginpunten en verwachte eindpunten. Welke databasegegevens controleer je naast het scherm?

Een fout vinden, herstellen en bewaren

Maak de fout eerst herhaalbaar

Debuggen is onderzoeken waarom de werkelijke uitkomst niet klopt. Verander niet meteen willekeurige regels code. Noteer eerst wat je deed, wat je verwachtte en wat er gebeurde. Zoek daarna waar de gegevens voor het eerst afwijken: in de invoer, het verzoek, de serverberekening of het scherm.

Foutverslag	Uitgewerkt voorbeeld
Titel	Alleen fluo levert onterecht twee punten op.
Begintoestand	Sara, open campagne, vandaag geen rij, totaal 0.
Stappen	Helm uit, Fluo aan, Opslaan.
Verwacht	1 dagpunt en totaal 1.
Werkelijk	Scherm toont totaal 2.
Onderzoek	Bekijk JSON en de bewaarde booleans. Controleer daarna formule en weergave.
Herstelcontrole	Voer alle vier itemcombinaties opnieuw uit; niet alleen het foutieve voorbeeld.

Git bewaart de geschiedenis van de code

Maak een commit na een klein werkend resultaat. Een boodschap zoals ‘Voorkom dubbele dagregistratie’ beschrijft meer dan ‘update’. Bekijk vóór commit welke bestanden veranderen. Neem geen echte leerlinggegevens, wachtwoorden of databasebestanden met persoonsgegevens op in het project.

Na een bugfix herhaal je de test die faalde en enkele verwante tests. Dat heet regressietesten: controleren dat de verbetering ander gedrag niet kapotmaakt. Een aangepaste puntenformule raakt bijvoorbeeld beide losse items én de combinatie ervan.

> **Zelf proberen ·** Schrijf een foutverslag voor twee keer dezelfde winnaar in één trekking. Geef ook een passende commitboodschap voor de herstelling.

Opleveren en blijven beheren

Een andere gebruiker moet ermee kunnen werken

Opleveren betekent dat iemand anders het project kan starten, begrijpen en gebruiken. Een app die alleen op jouw laptop werkt en waarvan niemand de stappen kent, is nog niet goed overgedragen. De oefenoplevering blijft klein, maar toont alle belangrijke onderdelen.

Bij oplevering	Wat leveren we?
Analyse	Eisen, regels, UML en schermschetsen die overeenkomen met de gebouwde versie.
Code	Een herkenbare repository zonder geheimen of echte leerlingdata.
Installatie	Welke runtime nodig is, waar configuratie staat en hoe fictieve data worden geladen.
Testverslag	Uitgevoerde tests met resultaat en eventuele open fouten.
Handleiding	Korte instructie voor medewerker en administrator.
Demo	Registreren, herhalen, corrigeren, filteren en trekken tonen.
Beheer	Hoe je een back-up maakt, een fout meldt en een nieuwe versie plaatst.

Hosting en onderhoud

Bij hosting verplaats je de webapp naar een bereikbare server. Zorg dat de database en configuratie niet publiek downloadbaar zijn. Controleer na de plaatsing de complete registratie op twee toestellen. Een lokaal geslaagde test is niet automatisch bewijs dat de serveromgeving correct is ingesteld.

Onderhoud betekent fouten herstellen, afhankelijkheden bijwerken en nieuwe wensen beoordelen. Een nieuwe wens begint weer bij analyse: wat verandert aan de betekenis en welke tests volgen? Maak vóór een belangrijke wijziging een back-up en controleer dat herstel echt mogelijk is.

> **Zelf proberen ·** Schrijf een handleiding van vijf korte handelingen voor een nieuwe medewerker. Waar vermeld je dat hij de opslagbevestiging moet afwachten?

Lestraject en concrete werkstukken

Een overzichtelijk project in duo’s

Dit voorstel omvat ongeveer 24 lesuren van 50 minuten. De precieze duur hangt af van de voorkennis. De leerkracht voorziet waar nodig een projectbasis, testaccounts en de API-start. Het doel is een klein prototype en de samenhang van het proces; de planning is geen raming voor een complete productie-installatie.

Lesblok	Werkstuk	Wat leerlingen uitleggen
1-4 · Begrijpen	Samenvatting, rollen, scope, eisen en regels.	Waarom de app nodig is en wat ze precies doet.
5-9 · Functioneel ontwerpen	Use-case- en activiteitendiagram, scenario en wireframes.	Hoe gebruiker en foutpaden door het proces gaan.
10-15 · Technisch ontwerpen	Klassenmodel, tabellen, JSON, API en sequentiediagram.	Welke gegevens waar staan en hoe onderdelen samenwerken.
16-20 · Klein bouwen	Klaslijst, opslag, totalen en trekking op fictieve data.	Hoe code de gekozen regels uitvoert.
21-24 · Controleren	Testverslag, bugfix, README en demonstratie.	Hoe ze weten dat het werkt en wat nog beperkt is.

Rollen in het team afwisselen

Laat bij ieder blok iemand uitwerken en iemand controleren, en wissel daarna. Beide leerlingen moeten het hele pad kunnen uitleggen. Een aparte diagrammaker die de code nooit ziet en een programmeur die de regels niet begrijpt, missen juist de samenhang die dit project wil aanleren.

Gebruik dezelfde zes fictieve leerlingen van analyse tot demo. Geef één leerling nul punten en twee leerlingen dezelfde achternaam. Daarmee worden meerdere controles vanzelf zichtbaar. Bij de trekking levert de leerkracht vaste testgetallen om het algoritme eerst zonder toevallige uitkomsten na te kijken.

> **Zelf proberen ·** Kies jullie eerste werkstuk en bepaal wie uitwerkt en wie controleert. Spreek één concreet resultaat af dat jullie op het einde van de les kunnen tonen.

Waarop beoordelen we het project?

Begrijpen telt mee, niet alleen code

Een leerling kan een werkend voorbeeld kopiëren zonder de werking te begrijpen. Daarom beoordeelt dit project ook de uitleg en het verband tussen de stappen. De puntenverdeling hieronder is een concreet voorstel voor de leerkracht, geen vastgelegd examenreglement.

Onderdeel	Punten op 100	Wat maakt het goed?
Functionele analyse	20	Duidelijke eisen, rollen, regels, scenario’s en foutpaden.
UML en schermen	20	Juiste symbolen en relaties; diagrammen en schermen passen bij de opdracht.
Technische analyse	20	Logisch gegevensmodel, begrijpelijk API-contract en correcte algoritmen.
Prototype	15	Een klein compleet kernpad met correcte gegevens en herhaalbare werking.
Testen en verbeteren	15	Verwachte uitkomsten, uitgevoerde controles en een onderbouwde bugfix.
Uitleg en samenwerking	10	Beide leerlingen kunnen keuzes verklaren en leveren een duidelijke overdracht.

Voorbeelden van bewijs

Vraag niet alleen ‘wat doet deze knop?’, maar ‘welke eis voert hij uit?’. Laat een leerling aanwijzen welke tabel een waarneming bewaart en waarom een herhaling geen extra punten mag geven. Laat hem een vast getal door het lotingsalgoritme volgen. Zulke vragen maken het begrip zichtbaar.

Een fout hoeft niet alle punten te kosten wanneer ze goed onderzocht en uitgelegd is. Andersom verdient een indrukwekkend scherm geen hoge score als de telling verkeerd is. Beoordeel telkens de afgesproken taak, de redenering en het bewijs dat erbij hoort.

> **Zelf proberen ·** Kies één ontwerpbeslissing die jullie kunnen verdedigen. Gebruik de vorm: ‘We kiezen ..., omdat ..., en controleren dit met ...’.

Voorbeeldantwoorden · Functionele analyse

Dit zijn mogelijke antwoorden op de belangrijkste opdrachten. Andere formuleringen kunnen even goed zijn wanneer ze logisch en consequent aansluiten bij de gekozen afspraken.

Punten en deelnemers

Pagina 12: Noor heeft vier punten en krijgt voor beide items twee erbij: zes totaal. Hetzelfde dagresultaat opnieuw bewaren houdt het op zes. Een leerling die niet deelneemt kan niet nieuw worden geregistreerd. Een deelnemer met nul punten kan wel geregistreerd worden, maar heeft nog geen lot voor een trekking.

Filteren en sorteren

Pagina 22: filteren op 3A laat leerlingen uit 3B weg. Sorteren op punten verandert alleen de volgorde van de gekozen lijst. Het aantal resultaten verandert daardoor niet. Een filter op leerjaar en klas moet een geldige combinatie opleveren.

Rollen en use cases

Pagina 15: F04 hoort bij registreren, dus bij de medewerker en via de generalisatie ook bij admin. F08 en F09 zijn beheer en trekking en horen alleen bij admin. De leerling is in dit ontwerp geen actor omdat hij de app niet bedient. Een leerlingportaal zou een extra actor en use case nodig maken.

Scenario en foutmelding

Sara met alleen fluo krijgt één dagpunt. Bij een gesloten campagne wordt niets bewaard en verschijnt een begrijpelijke melding. ‘Kies maximaal 3 winnaars’ is bruikbaarder dan ‘ERROR 422’, omdat ze zegt wat de gebruiker moet veranderen.

Campagnestatus

Pagina 41: een al geopend leerlingdetail geeft geen uitzonderingsrecht. De actuele serverstatus Gesloten bepaalt dat Opslaan geweigerd wordt. De app vernieuwt die status en laat de medewerker geen nieuwe punten bevestigen.

> Bij het nabespreken telt vooral de redenering. Vraag steeds welke regel of eis het antwoord verklaart. Zo leer je verbanden leggen in plaats van alleen een juist cijfer geven.

Voorbeeldantwoorden · Technische analyse

Objecten, tabellen en totalen

Pagina 29: Noor met drie waarnemingen heeft één Leerling-object en drie Registratie-objecten. In de database zijn dat één leerlingrij en drie registratierijen. In registratie verwijzen leerlingId, campagneId en klasId naar andere tabellen. Helm is een waarde binnen de rij en dus geen vreemde sleutel.

Pagina 33: Noor krijgt 2 + 1 + 2 = 5 punten. Milan krijgt 1 + 0 = 1. Sara krijgt nul ondanks het ontbreken van registraties. Zonder klaswissels is het klastotaal zes. De LEFT JOIN voorkomt dat Sara uit de lijst verdwijnt.

UML en communicatie

Pagina 30: bij drie kandidaten en twee winnaars krijgen de kandidaten samen winnaarRang 1, 2 en geen waarde. Pagina 37: bij een verkeerde versie antwoordt de server met een conflict na het lezen van de huidige toestand. De bewaarstap en databasebevestiging van een nieuwe opslag mogen dan niet plaatsvinden.

Gewogen trekking narekenen

Pagina 40: het eerste getal 8 kiest B. Daarna blijven A met zeven punten en C met twee over. Het nieuwe totaal is negen, A heeft 1-7 en C 8-9. Het volgende getal 9 kiest C. B en C winnen; B kan niet opnieuw worden gekozen.

Code en beveiliging

Pagina 44: de invoer zijn klasId en campagneId. De functie maakt een verzoek, controleert status en leest JSON. Een lege geldige lijst is een succesvol antwoord met geen leerlingen; netwerkverlies is een fout zonder betrouwbaar nieuw resultaat. Een zelf meegestuurde adminrol is geen bewijs: de server haalt de echte rol uit de aangemelde identiteit.

> Controleopdracht voor de hele cursus: volg één waarneming van bedrijfsregel via scherm, JSON, API en tabel naar totaal en test. Als je dat pad kunt verklaren, heb je de samenhang van softwareontwikkeling begrepen.

Een korte begrippenlijst

Begrip	Betekenis in dit project
Functionele analyse	Wat de app voor de gebruiker moet doen.
Technische analyse	Hoe de onderdelen dat samen uitvoeren.
Eis / requirement	Een concrete afspraak die je kunt controleren.
Bedrijfsregel	Betekenisvolle afspraak, zoals maximaal twee dagpunten.
User story	Wie wil welke taak uitvoeren en waarom?
Use case	Een taak met actor, voorwaarden, verloop en resultaat.
UML	Een afgesproken tekentaal voor softwaremodellen.
Wireframe	Eenvoudige schets van informatie en bediening.
Frontend / backend	Scherm en interactie / serverlogica en controle.
API / JSON	Communicatieafspraak / formaat van uitgewisselde gegevens.
Primaire / vreemde sleutel	Unieke rij-identiteit / verwijzing naar een andere tabel.
Transactie	Gegevenswijzigingen die samen slagen of teruggedraaid worden.
Versieconflict	De rij veranderde nadat de gebruiker ze gelezen had.
Algoritme / pseudocode	Stappen voor een oplossing / leesbare beschrijving daarvan.
Gewogen zonder teruglegging	Meer punten geven meer kans; geselecteerde leerling verdwijnt.
Regressietest	Controle dat een verbetering geen ander gedrag kapotmaakt.
Deployment	De app in een bruikbare serveromgeving plaatsen.

Je hoeft niet alle woorden uit het hoofd op te zeggen. Je moet ze kunnen gebruiken om je ontwerp te verklaren. Wijs bijvoorbeeld in jullie project aan welke gegevens door de API gaan en welke unieke sleutel dubbele dagregistraties voorkomt.

Bronnen en laatste controle

Waarop is dit lesmateriaal gebaseerd?

De projectwerking komt uit het aangeleverde verlanglijstje voor de fluo-app en de eerder gekozen eenvoudige afspraken. Dit dossier past de functionele en technische analyse aan voor het 5e middelbaar en voegt echte UML-modellen, kleine opdrachten en voorbeeldantwoorden toe. Het is lesmateriaal met een uitgewerkte analyse, geen reeds gebouwde productie-app.

[S1] Object Management Group - UML 2.5.1. Referentie voor de modelleertaal. De diagrammen gebruiken een beperkte set symbolen voor use cases, activiteiten, klassen, sequenties en toestanden.
omg.org · Unified Modeling Language

[S2] MDN - Using the Fetch API. Achtergrond voor het JavaScript-verzoek, HTTP-statuscontrole en het lezen van JSON.
developer.mozilla.org · Using Fetch

[S3] PHP Manual - random_int. Voorbeeld van een uniforme serverbron voor gehele willekeurige getallen.
php.net · random_int

[S4] SQLite - CREATE TABLE en Foreign Key Support. Referentie voor tabellen, beperkingen en het activeren van vreemde-sleutelcontrole.
sqlite.org · CREATE TABLE
sqlite.org · Foreign Key Support

Wat leveren leerlingen na dit traject op?

Een begrijpelijke opdrachtomschrijving, duidelijke eisen en regels, vijf soorten UML-diagrammen, schermschetsen, een eenvoudig gegevensmodel, API-afspraken, algoritmen, een klein prototype en aantoonbare tests. De demo verbindt die resultaten: dit was de vraag, zo hebben we ze uitgewerkt en zo controleren we de oplossing.

> Laatste vraag: kun je bij één registratie uitleggen waarom elke stap nodig is? Van de medewerker die iets waarneemt tot de leerling die punten krijgt, en van die punten tot een verklaarbare trekking.

Bronnen geraadpleegd op 4 oktober 2026. Alle persoonsnamen, datasets, leerplanning en beoordelingspunten zijn voorbeelden voor dit leerproject.